

Virtual Memory

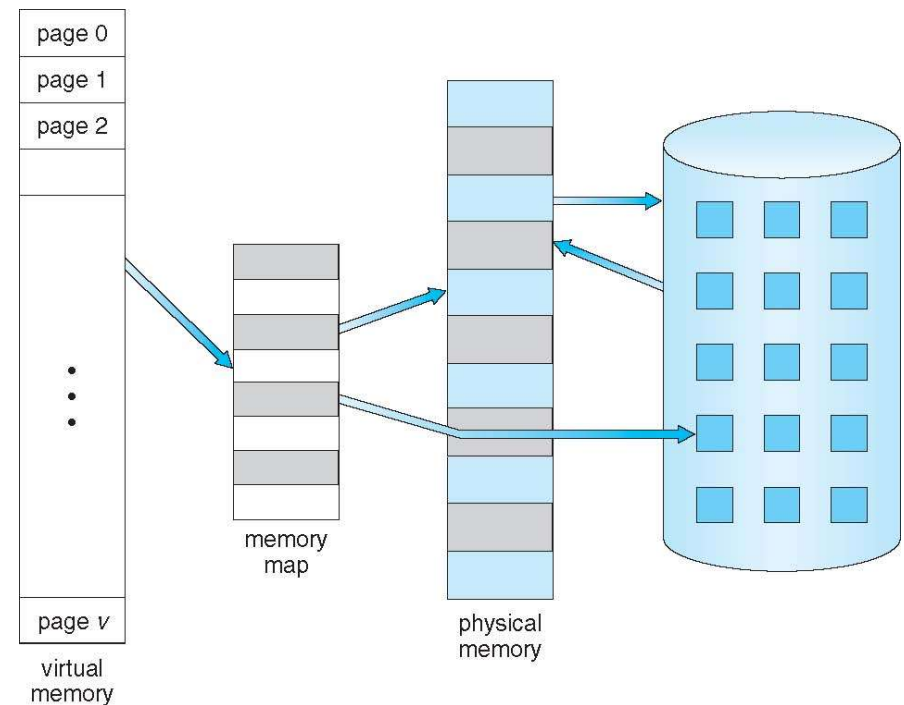


Virtual Memory

- ❑ Code needs to be in memory to execute, but entire program rarely used
 - ❑ Error code, unusual routines, large data structures
- ❑ No need to load all pieces of a process in main memory – VM
- ❑ Advantages
 - ❑ Program no longer constrained by limits of physical memory
 - ▶ A process may be larger than all of main memory
 - ❑ Each program takes less memory while running → more programs run at the same time (in Ready Queue)
 - ▶ Increased CPU utilization and throughput with no increase in response time or turnaround time
 - ❑ Less I/O needed to load or swap programs into memory → each user program runs faster

Virtual Memory

- ❑ **Virtual memory** separation of logical memory as perceived by users from physical memory
 - ❑ Only part of the program needs to be in memory for execution
 - ❑ Logical address space can be much larger than physical address space
- ❑ **Virtual address space** – logical view of how process is stored in memory
 - ❑ Usually start at address 0, contiguous addresses until end of space
 - ❑ physical memory organized in page frames
 - ❑ MMU must map logical to physical

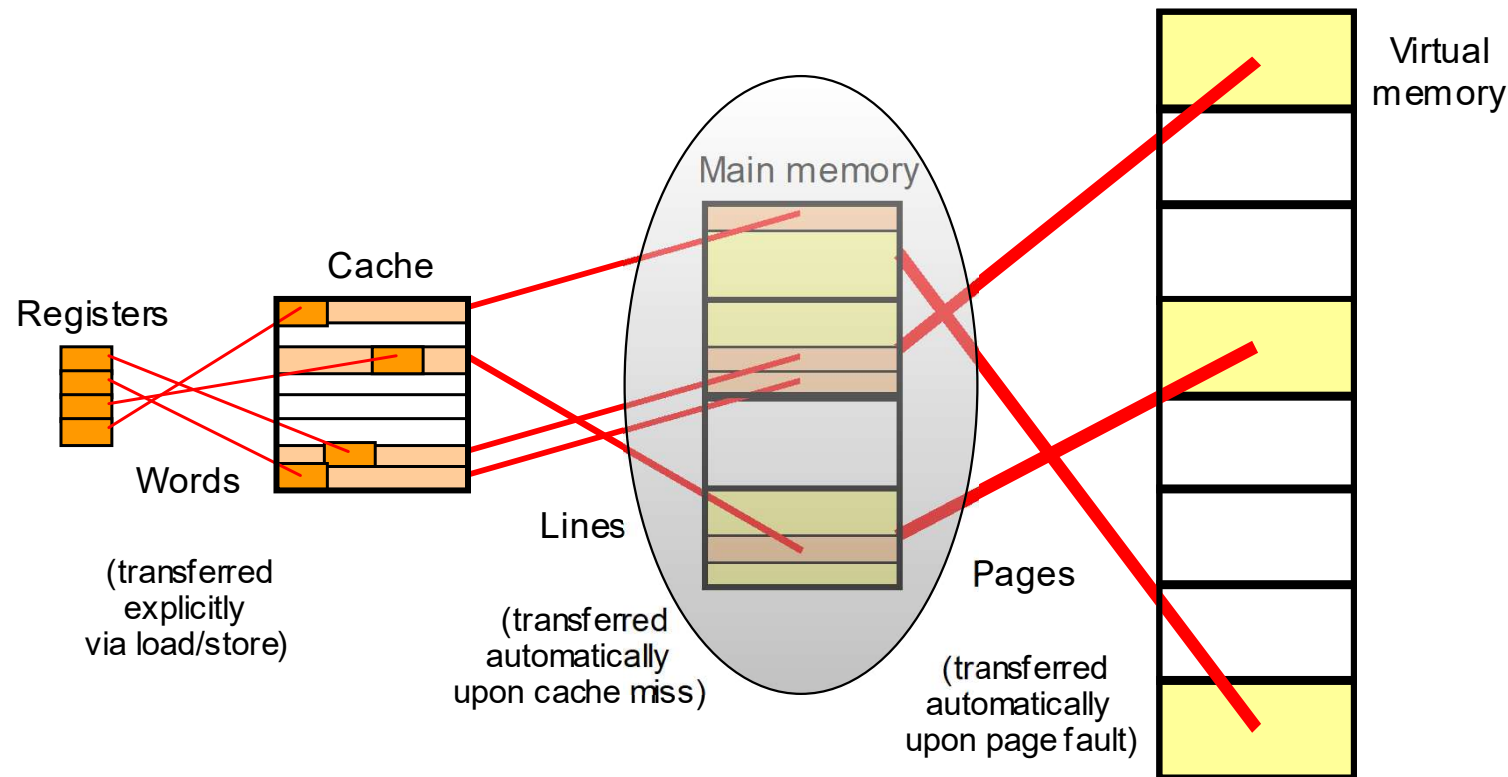


Memory Hierarchy

Cache memory:
provides illusion of
very high speed

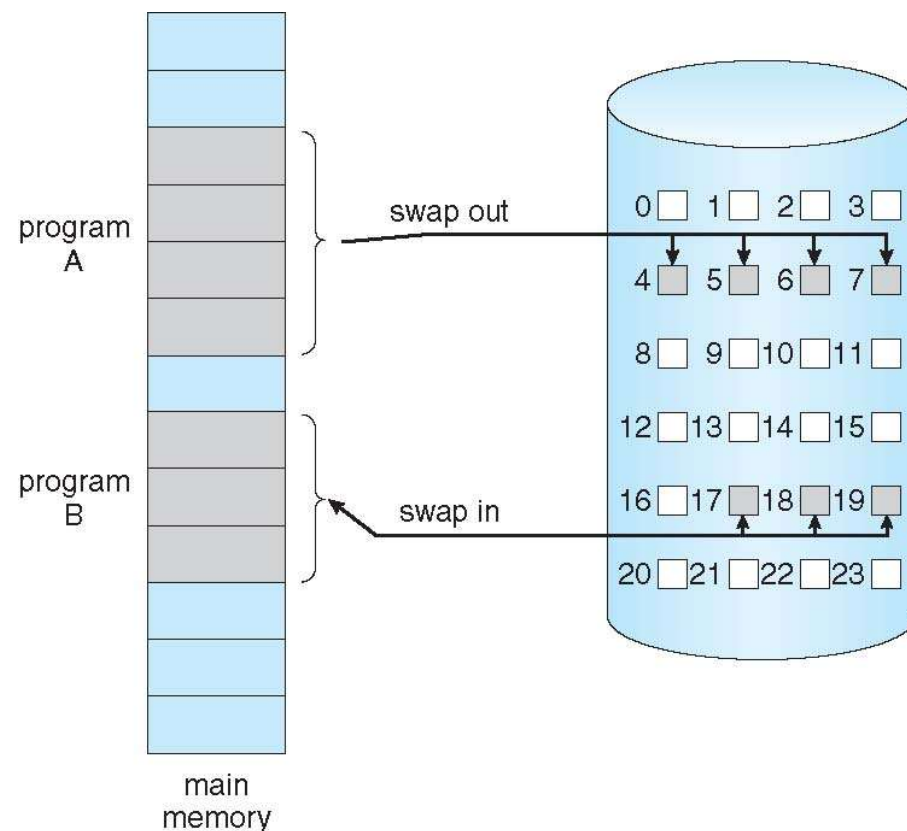
Main memory:
reasonable cost,
but slow & small

Virtual memory:
provides illusion of
very large size



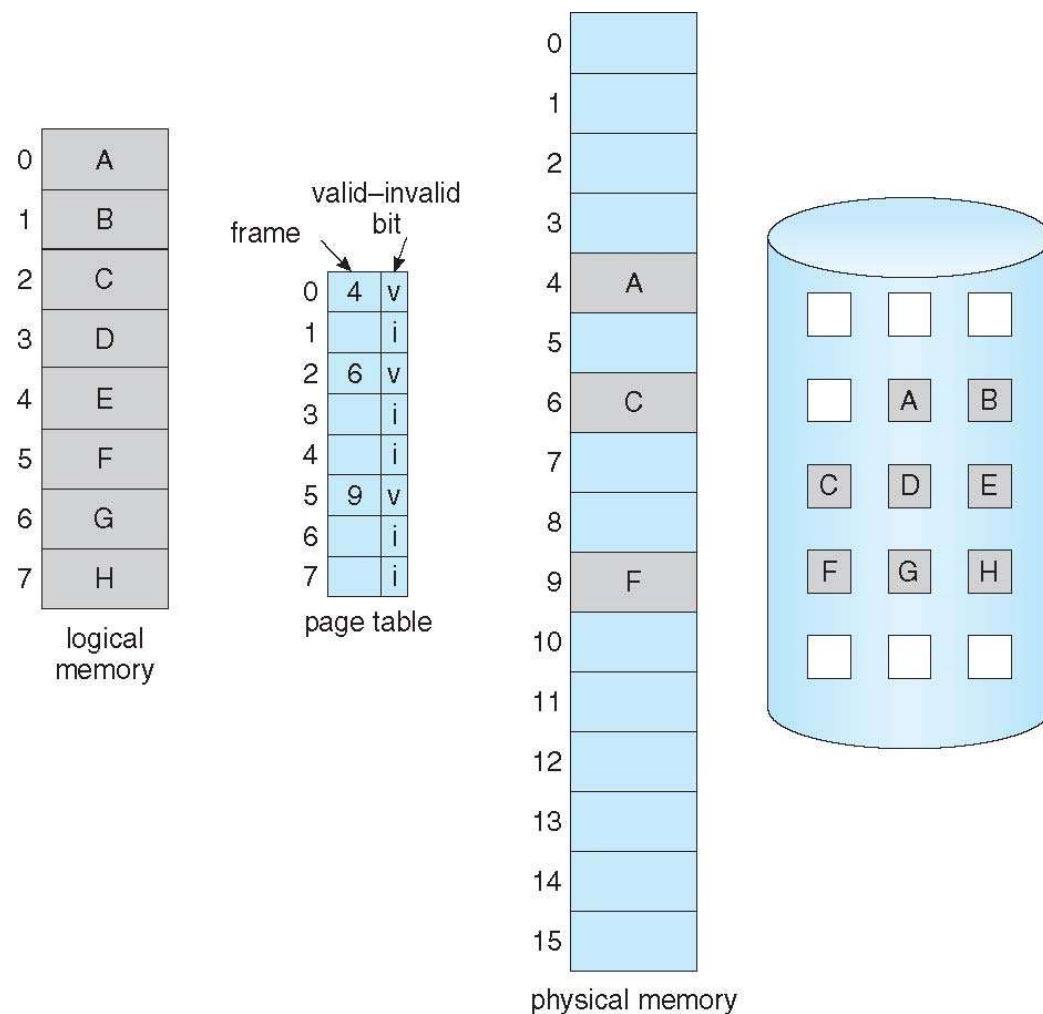
Demand Paging

- ❑ **demand paging** pages are loaded only when they are demanded during program execution
- ❑ Paging is similar to swapping
 - ❑ **Swapper**: entire process
 - ❑ **Pager**: swaps pages
- ❑ When a Page is needed $\Rightarrow$ reference to it
 - ❑ invalid reference $\Rightarrow$ trap
 - ❑ In-memory $\Rightarrow$ proceed
 - ❑ not-in-memory $\Rightarrow$ bring to memory



Valid-Invalid Bit

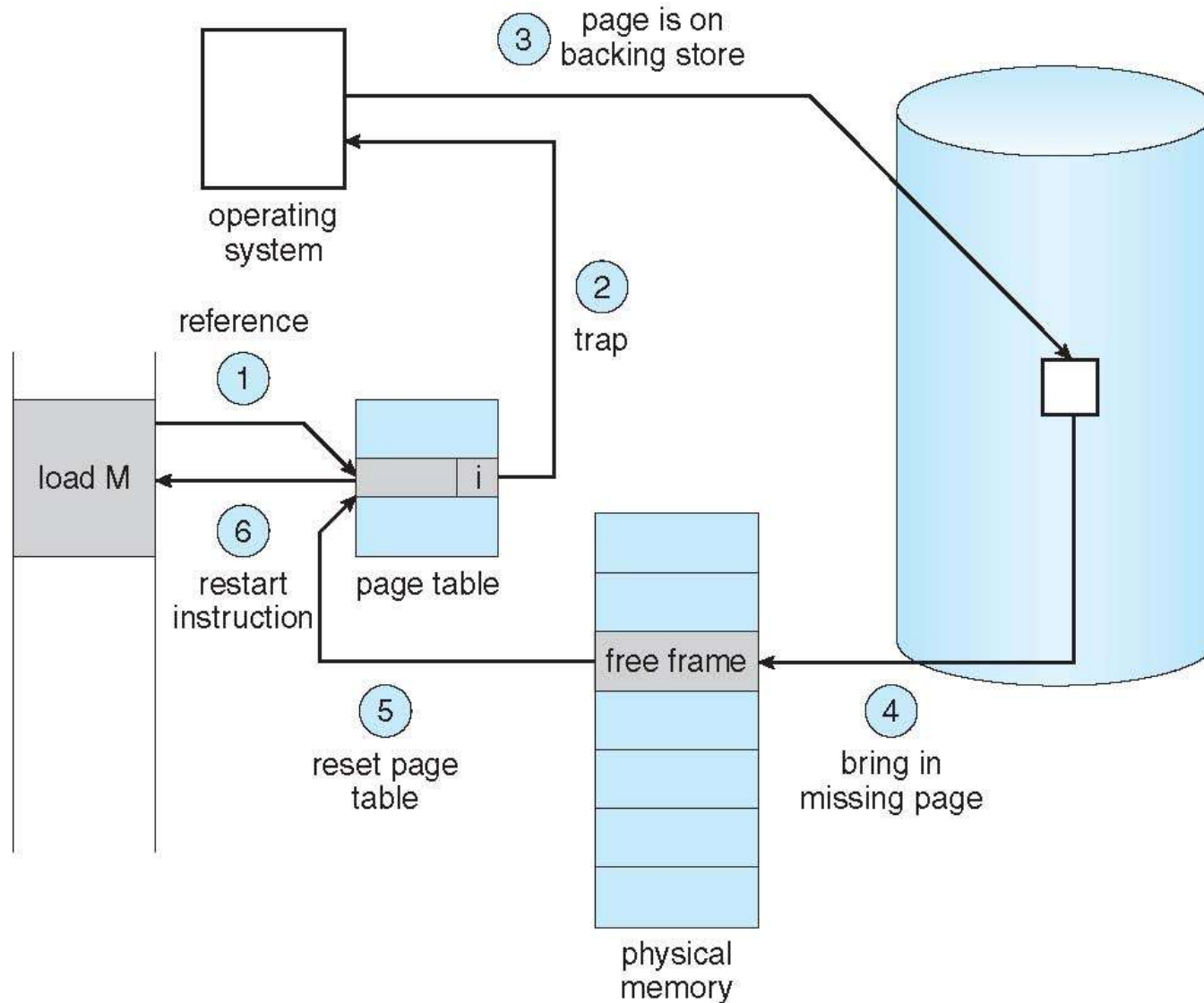
- With each page table entry a valid–invalid bit is associated (**v** $\Rightarrow$ in-memory – **memory resident**, **i** $\Rightarrow$ not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- During MMU address translation, if valid–invalid bit in page table entry is **i** $\Rightarrow$ **page fault**



Page Fault

- what happens if the process tries to access a page that was not brought into memory?
 - Access to a page marked invalid causes a **page fault**.
- The **paging hardware**, in translating the address through the page table, will notice that the invalid bit is set, causing a **trap**
- The **procedure** for handling the page fault
 1. OS checks the PCB to determine whether the reference valid/invalid memory reference
 2. If invalid, terminate the process. If valid; page it in.
 3. find a free frame
 4. schedule a disk operation to read the desired page into the newly allocated frame.
 5. When the disk read is complete, modify the page table to indicate that the page is now in memory.
 6. restart the instruction that was interrupted by the trap.

Steps in Handling a Page Fault



Execution of a Process

- OS brings into main memory few pieces of the program
- **Resident set** - portion of process that is in main memory
- Process executes as long as memory references are in resident set – determined from page/segment tables
- When a reference to a logical address is not in memory
 - An **interrupt** is generated signaling a **page fault**
 - OS places the process in a **blocking state**
 - I/O request is started to get the page from disk
 - OS Dispatch another process
 - An interrupt is issued when disk I/O is complete which causes the OS to place the process in the **Ready Queue**

Principle of Locality

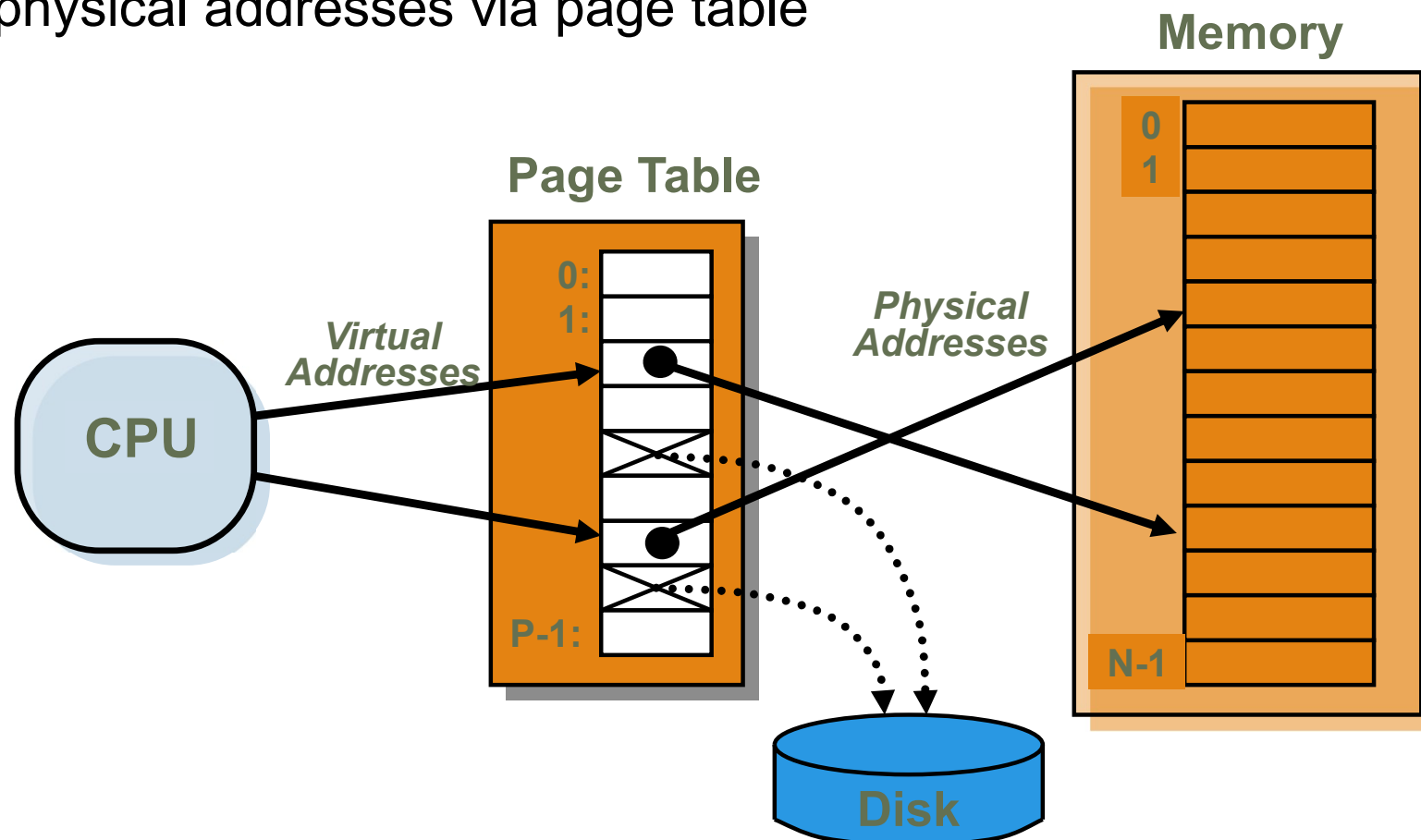
- Program and data **references** within a process tend to **cluster**
 - Only a few pieces of a process will be needed over a short period of time
 - ▶ More processes can be maintained in memory
 - ▶ Time is saved because unused pieces are not swapped
- Therefore, ***principle of locality*** can be used to make virtual memory to work efficiently
- It is possible to make intelligent guesses about which pieces will be needed in the future, which avoids thrashing

Thrashing

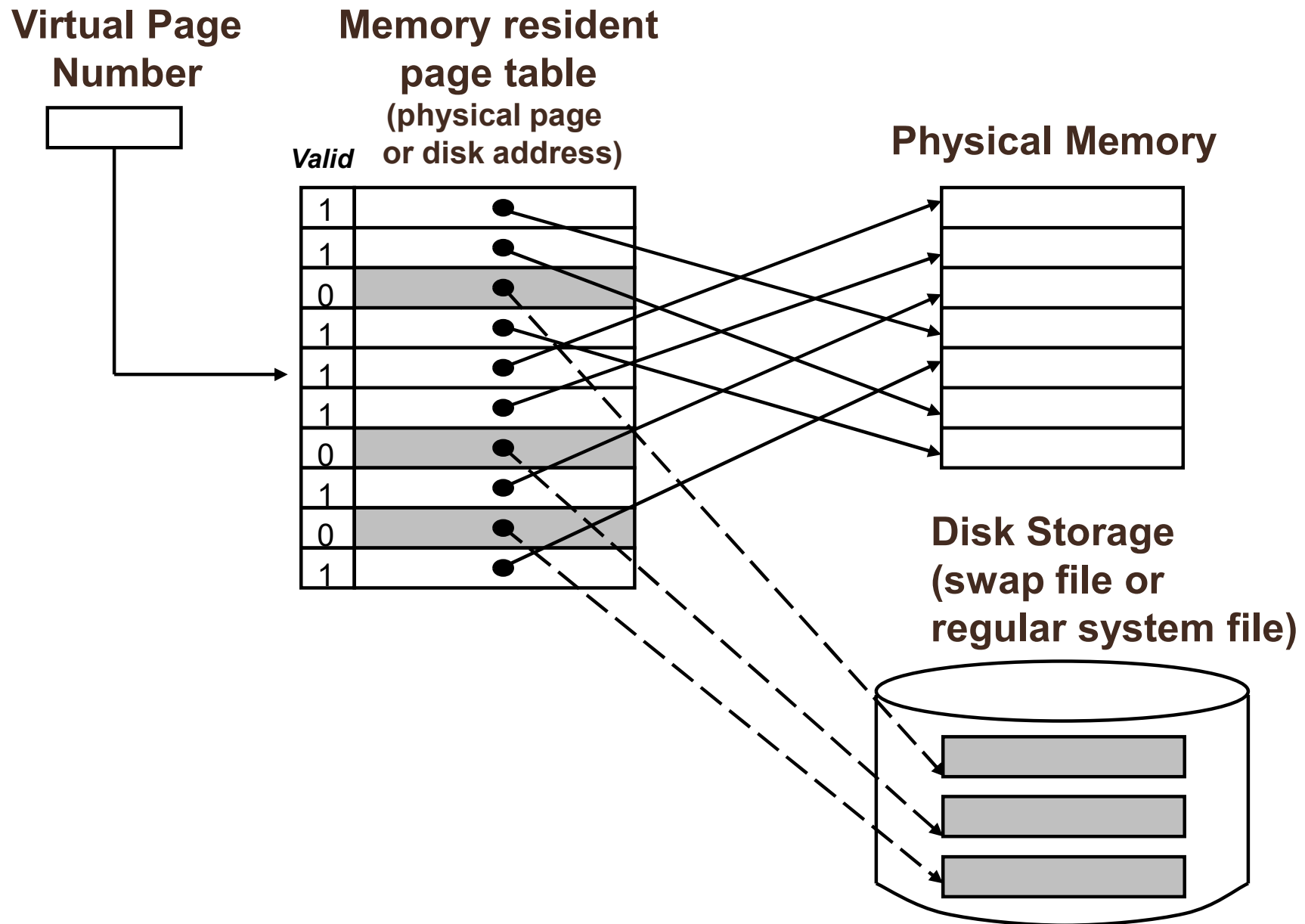
- Usually all of main memory will be occupied with process pieces
- when OS brings a piece in, it must throw one out – which one?
 - If it throws out a piece, just before it is used, then it will have to get it again immediately
 - Too much of this causes **Thrashing**
- **Thrashing**
 - Swapping out a piece of a process just before that piece is needed
 - The processor spends most of its time swapping pieces rather than executing user instructions
 - OS has to avoid this!

A System with Virtual Memory (paging)

- Each process has its own **page table**
- Each page table entry contains the **frame number** of the corresponding page in main memory or the disk address
- **Address Translation**: Hardware converts virtual addresses to physical addresses via page table



Page Tables



Paging

Virtual Address



Page Table Entry



- Each process has its own page table
- Each Page Table Entry (PTE) contains the **frame number** of the corresponding page in main memory
- Two extra bits are needed to indicate:
 - **P** whether the page is in main memory or not
 - **M** Whether the contents of the page has been altered since it was last loaded (**dirty**). If no change has been made, the page does not have to be written to the disk when it needs to be swapped out
- Control bits: Accessed, various protection-related bits

Support Needed for Virtual Memory

- **Hardware** must support paging and/or segmentation
 - Hardware support for *address translation* is critically needed for paging
 - Software/Hardware support to handle *page faults*
- **OS** must manage *moving pages efficiently* between secondary memory and main memory
 - What if no room in main memory?
 - Which frame to replace?

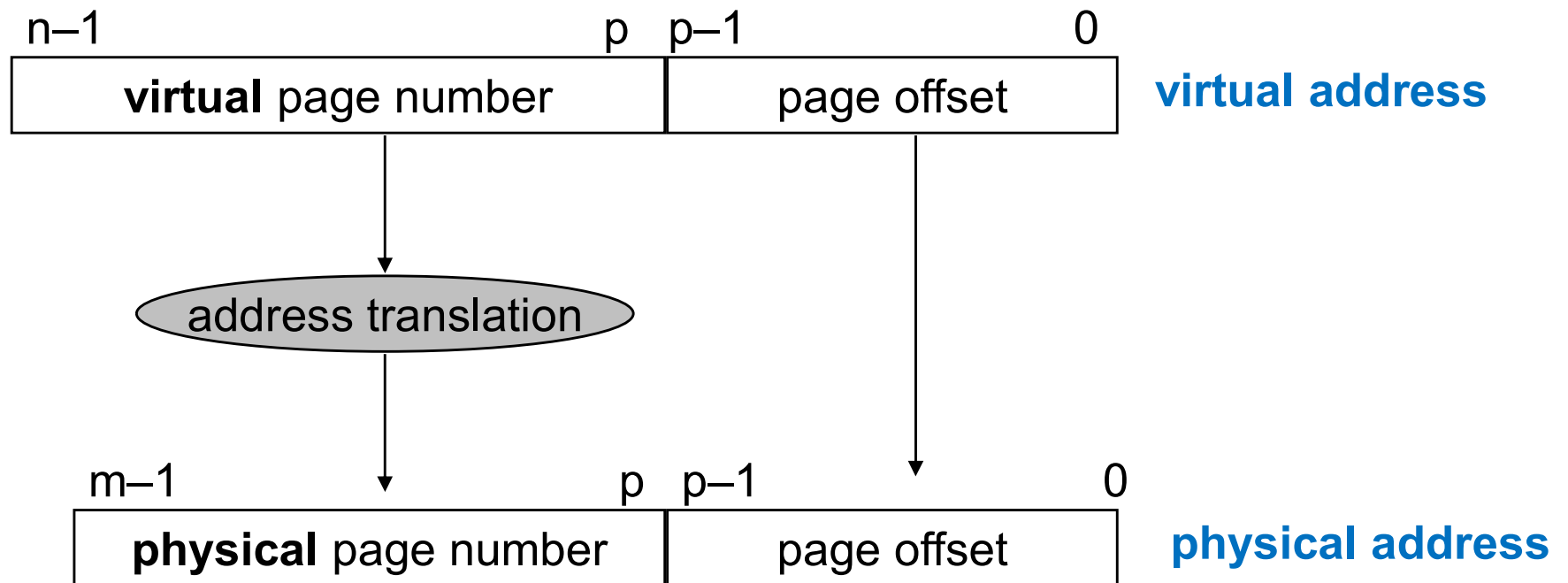
VM Address Translation

Parameters

$P = 2^p = \text{page size (bytes)}$

$N = 2^n = \text{Virtual address limit}$

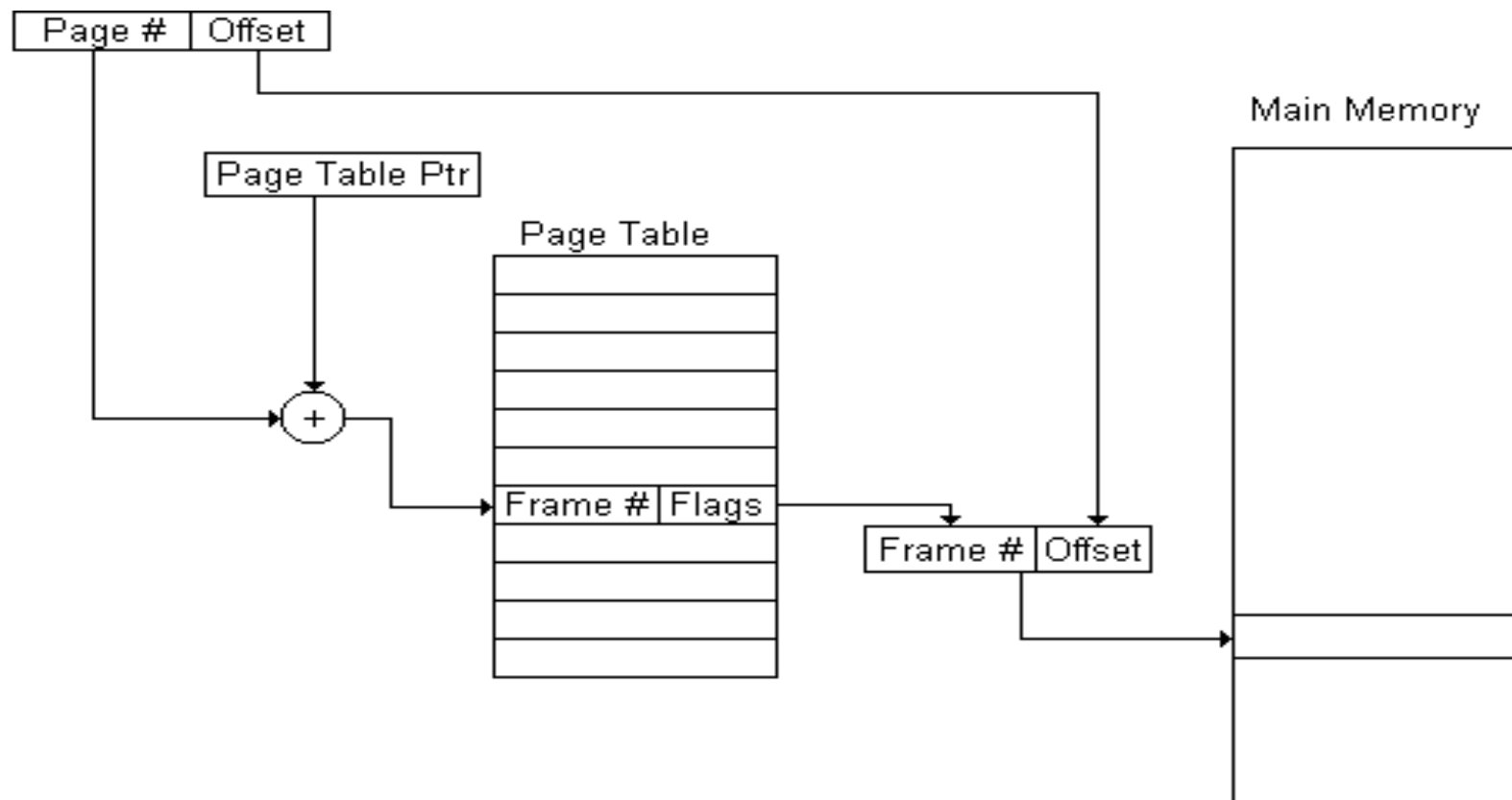
$M = 2^m = \text{Physical address limit}$



Page offset bits don't change as a result of translation

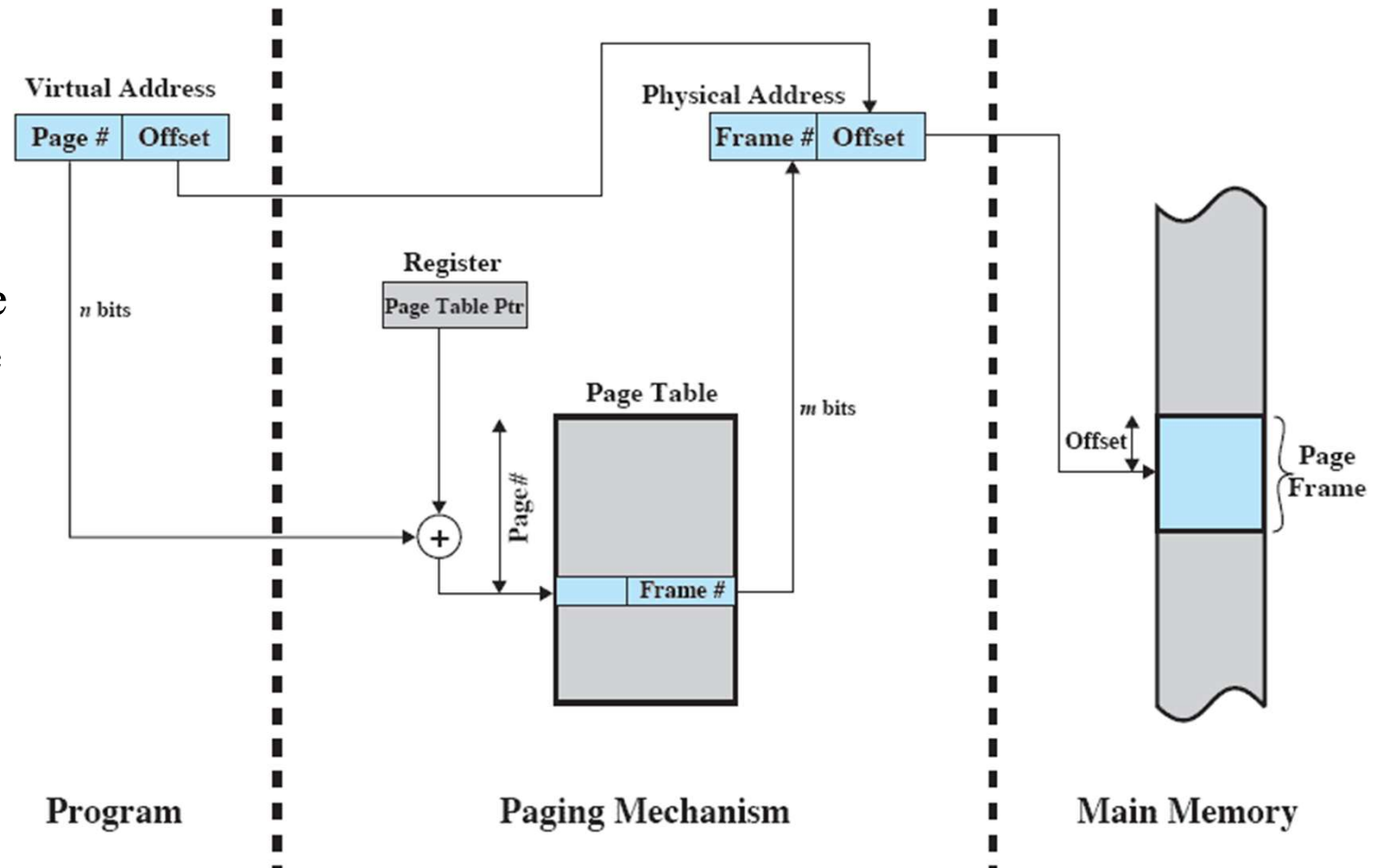
Paging Hardware

- Use page number as an index into the page table, which then contains the physical frame holding that page
- Typical Flag bits: Present, Accessed, Modified, various protection-related bits



Address Translation: Hardware Implementation

- register holds **the starting address of the page table**
- page number of a virtual address is used to index that table and look up the corresponding frame number.
- combined with the offset portion of the virtual address to produce the desired real address.
- Typically, the page number field is longer than the frame number field ($n > m$)



Structure of the Page Table

- Memory structures for paging can get huge using straight-forward methods
 - Consider a 32-bit logical address space
 - Page size of 4 KB (2^{12})
 - Page table would have 1 million entries ($2^{32} / 2^{12}$)
 - If each entry is 4 bytes -> 4 MB of physical address space / memory for page table alone
 - ▶ That amount of memory used to cost a lot
 - ▶ Don't want to allocate that **contiguously** in main memory
- There are different techniques for structuring the page table
 - **Hierarchical Paging**
 - **Hashed Page Tables**
 - **Inverted Page Tables**

Multi-Level Page Table

- Given:
 - 32-bit address space ($2^{32}=4\text{GB}$ of Virtual Address Space)
 - page size = 4KB (2^{12})
 - 4-byte PTE
- Problem:
 - 4GB is composed of 2^{20} pages
 - Page table = $2^{20} * 4$ bytes – that is 4 MB page table!
 - Too much to devote to page table
- Common solution
 - multi-level page tables e.g. 2-level table
 - Root Page table: 1024 entries, each of which points to a Level 2 page table – always in RAM
 - User Page Table: 1024 entries, each of which points to a page – stored in VM

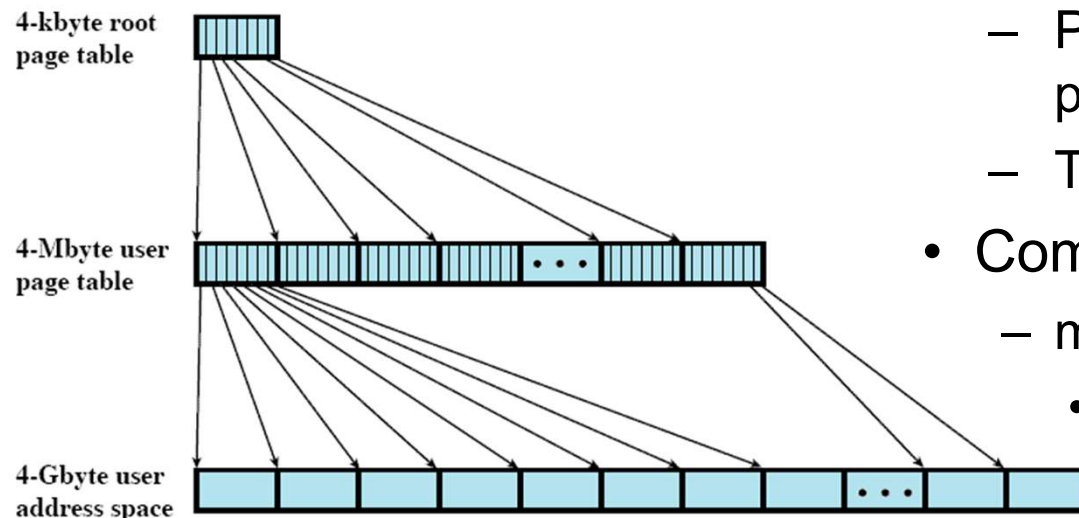
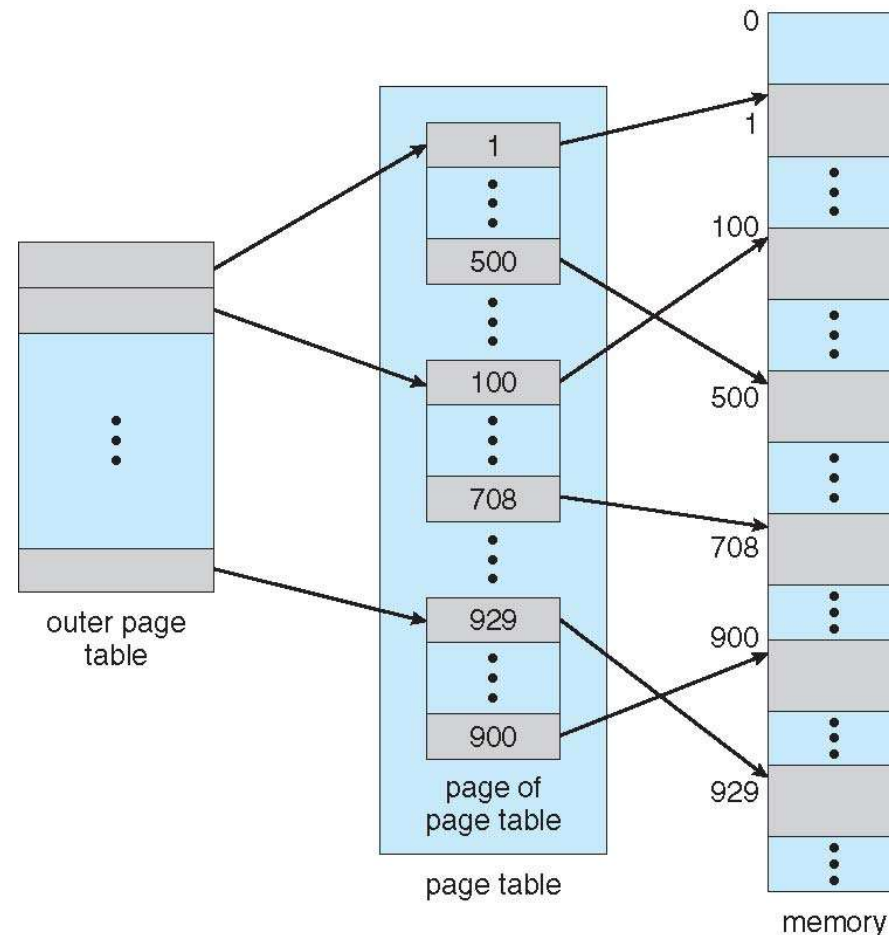


Figure 8.4 A Two-Level Hierarchical Page Table

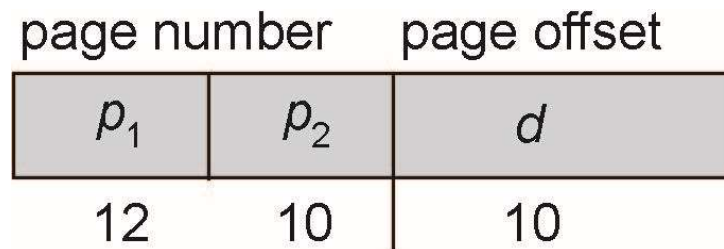
Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table



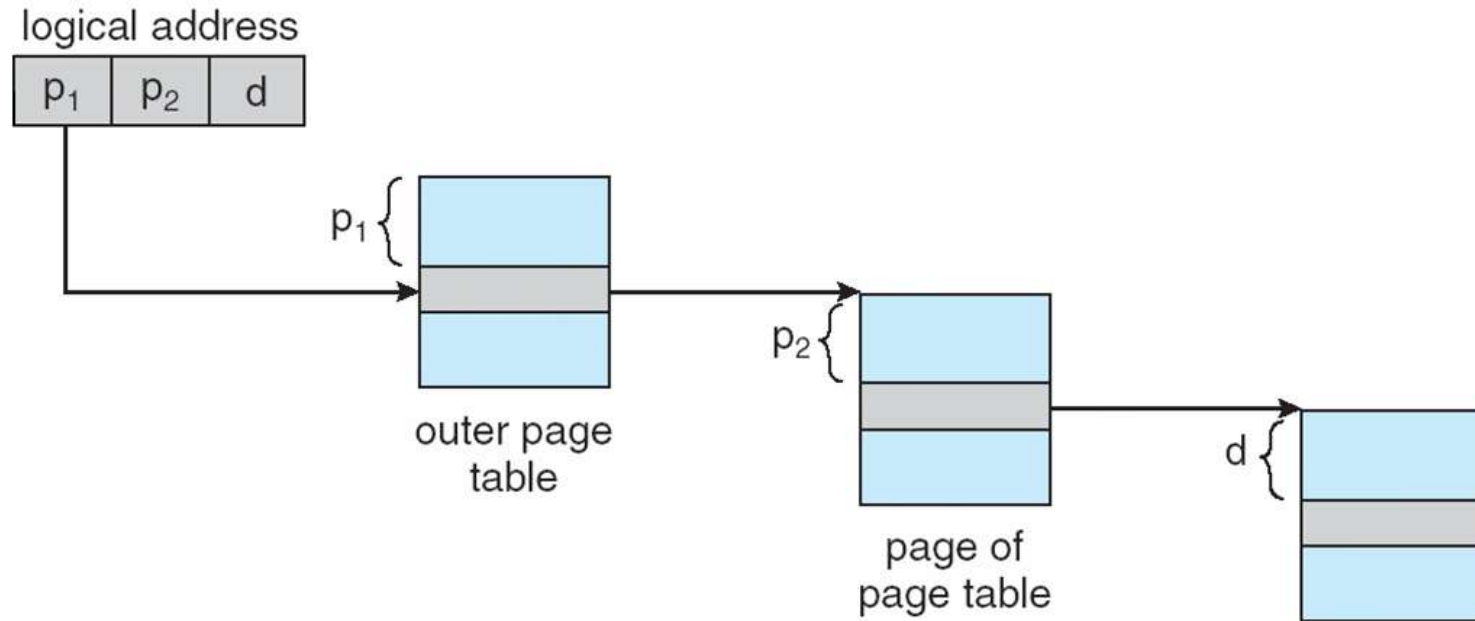
Two-Level Paging Example

- A logical address (on 32-bit machine with 1K page size) is divided into:
 - a page number consisting of 22 bits
 - a page offset consisting of 10 bits
- Since the page table is paged, the page number is further divided into:
 - a 12-bit page number
 - a 10-bit page offset
- Thus, a logical address is as follows:



- where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the inner page table
- Known as **forward-mapped page table**

Address-Translation Scheme



Address Translation: Hierarchical page table

- The **root page** always remains **in main memory**
- The first 10 bits of a virtual address are used to index into the root page to find a PTE for a page of the user page table.
- If that page is **not** in main memory, a page fault occurs
- If that page **is** in main memory, then the next 10 bits of the virtual address index into the user PTE page to find the PTE for the page that is referenced by the virtual address.
- Thus, if the length of the root page is X, and if the maximum length of a page table is Y, then a process can consist of up to $X * Y$ pages

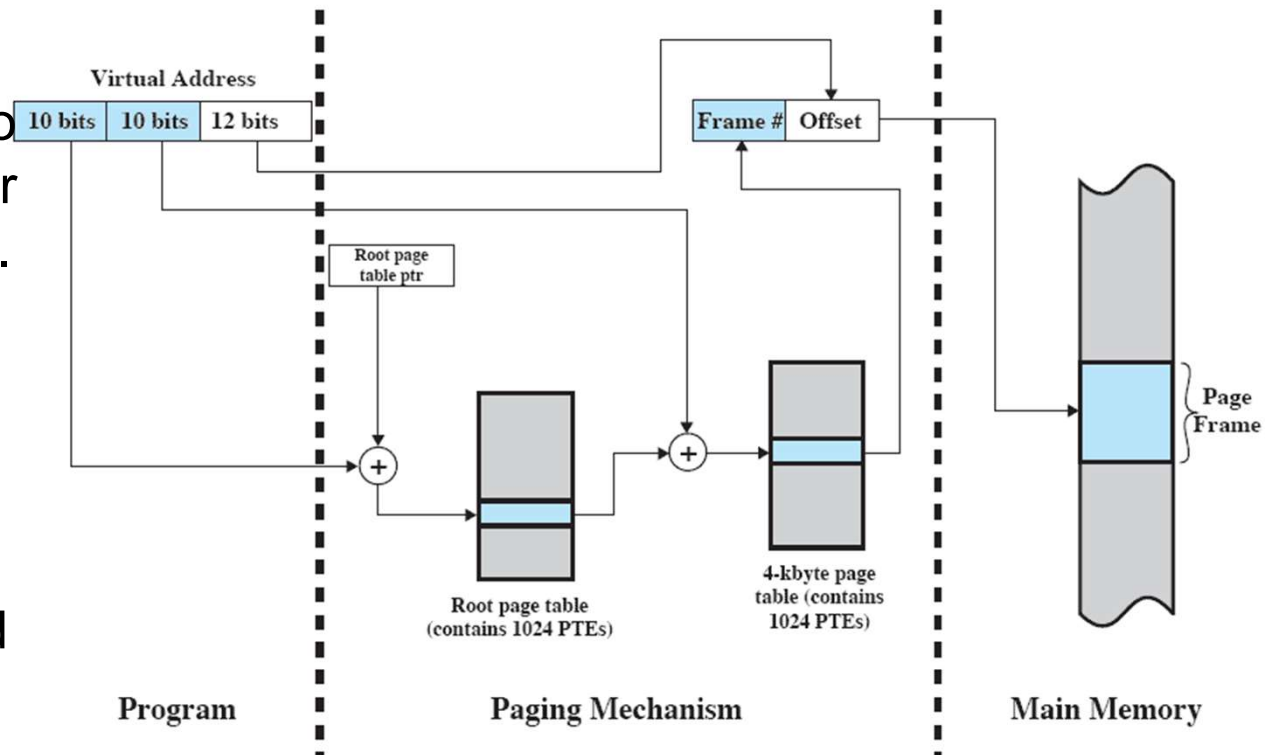
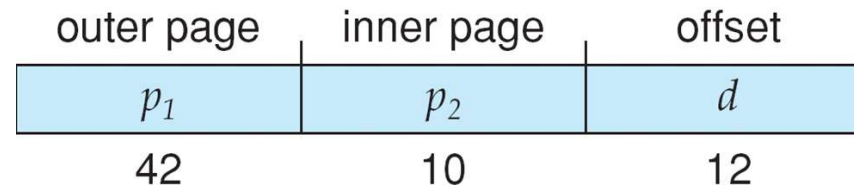


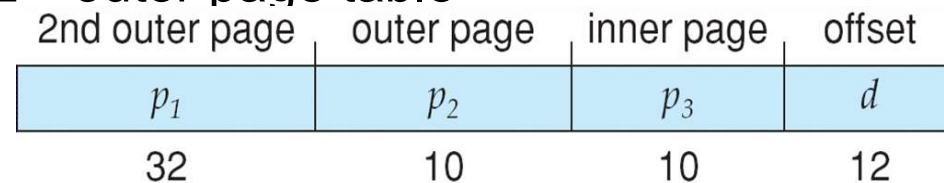
Figure 8.5 Address Translation in a Two-Level Paging System

64-bit Logical Address Space

- If page size is 4 KB (2^{12}), Then page table has 2^{52} entries
- With Two-level paging scheme, inner page tables could be 2^{10} 4-byte entries. Address would look like



- Outer page table has 2^{42} entries or 2^{44} bytes
- Three level paging → add a 2nd outer page table

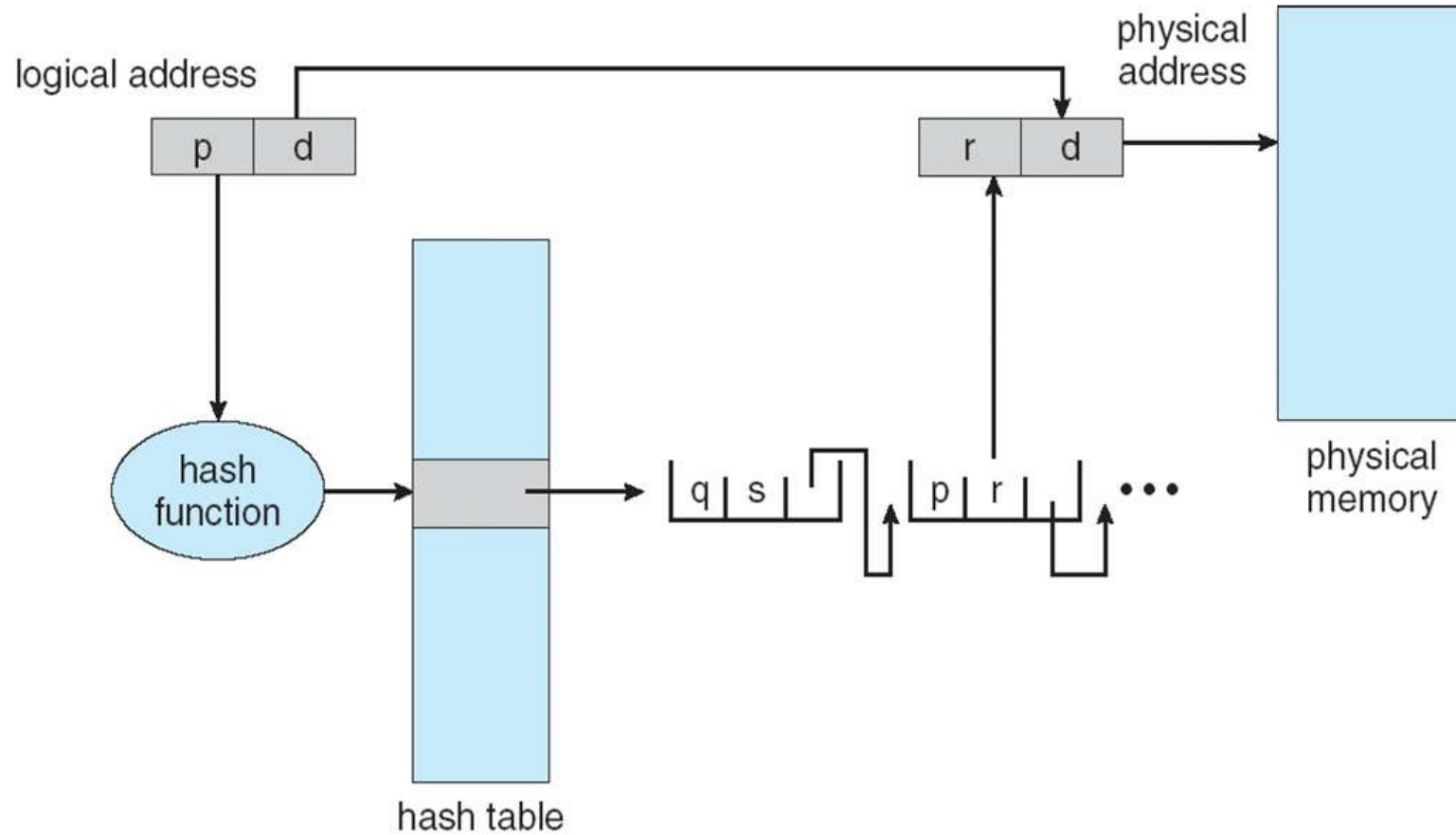


- But the 2nd outer page table is still 2^{34} bytes (**16 GB**) in size
- And possibly 4 memory access to get to one physical memory location
- Four level paging .. Etc ...
- 64-bit UltraSPARC would require seven levels of paging—a prohibitive number of memory accesses—to translate each logical address
- Hierarchical page tables are not generally appropriate

Hashed Page Tables

- ❑ Common in address spaces > 32 bits
- ❑ The virtual page number is hashed into a page table
 - ❑ This page table contains a chain of elements hashing to the same location (to handle collisions)
- ❑ Each element contains
 - (1) the virtual page number
 - (2) the value of the mapped page frame
 - (3) a pointer to the next element
- ❑ Virtual page numbers are compared in this chain searching for a match. When a match is found, the corresponding physical frame is extracted
- ❑ Variation for 64-bit addresses is **clustered page tables**
 - ❑ Similar to hashed but each entry refers to several pages (such as 16) rather than 1
 - ❑ Especially useful for **sparse** address spaces (where memory references are non-contiguous and scattered)

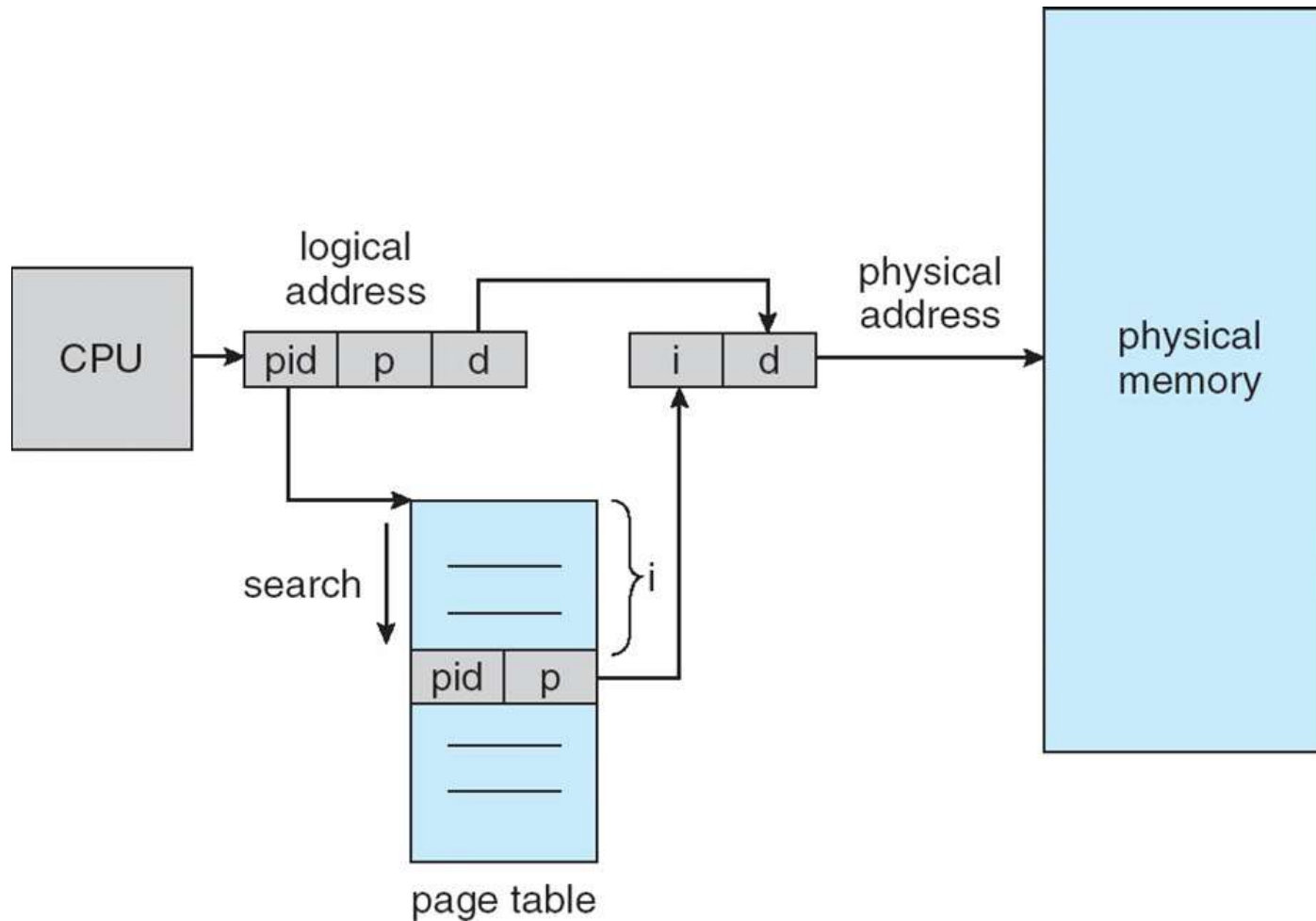
Hashed Page Table



Inverted Page Table

- Rather than each process having a page table and keeping track of all possible logical pages, track all physical pages
- One entry for each real page of memory
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page

Inverted Page Table Architecture



Inverted Page Table

- **Adv**: Decreases memory needed to store each page table
- **Disadv**: increases time needed to search the table when a page reference occurs
- Use hash table to limit the search to one — or at most a few — page-table entries
 - Each entry in the page table includes:
 - ▶ Page number
 - ▶ Process identifier: The process that owns this page.
 - ▶ Control bits: includes flags, such as valid, referenced, etc
 - ▶ Chain pointer: the index value of the next entry in the chain.

Inverted Page Table

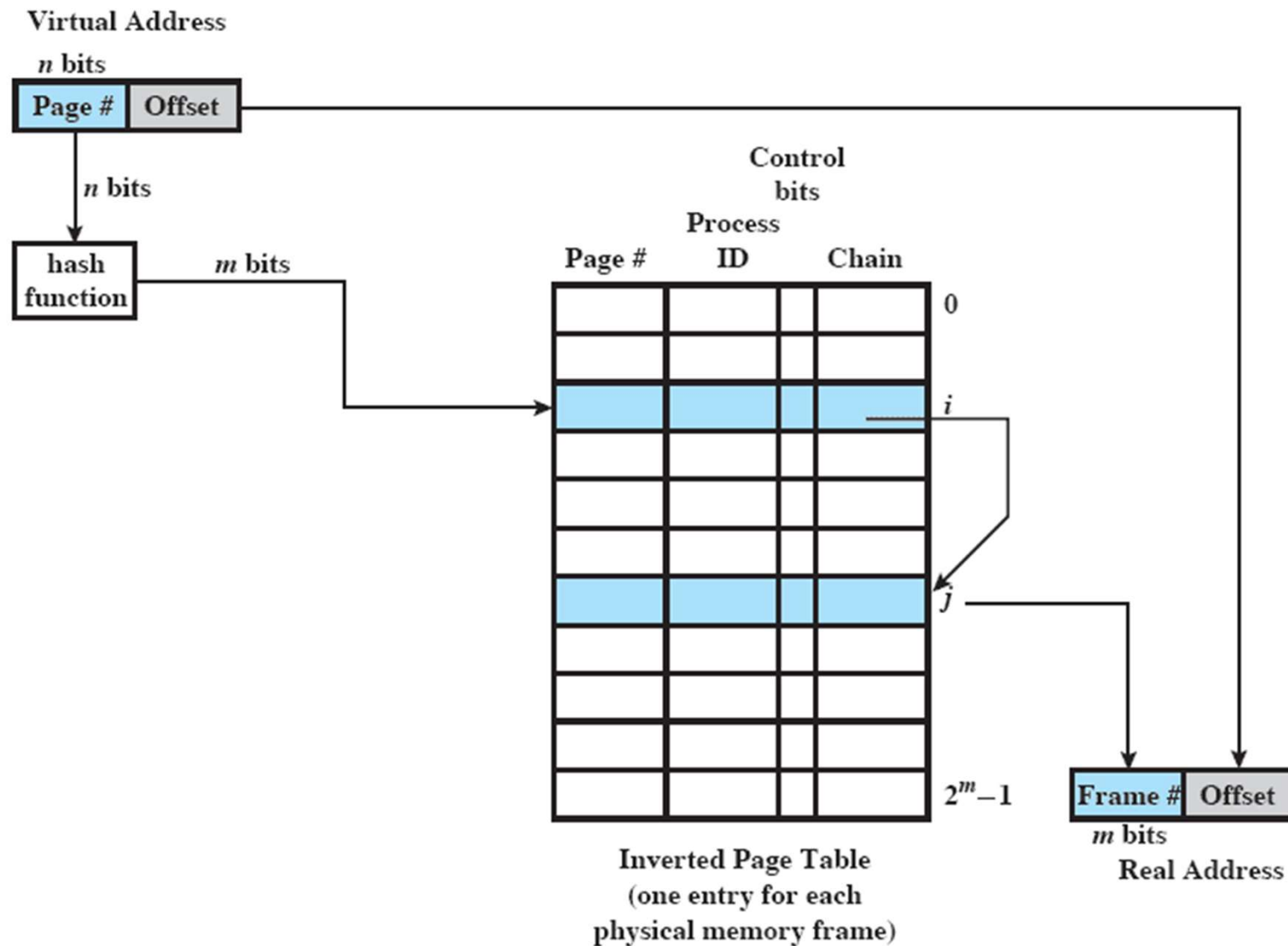


Figure 8.6 Inverted Page Table Structure

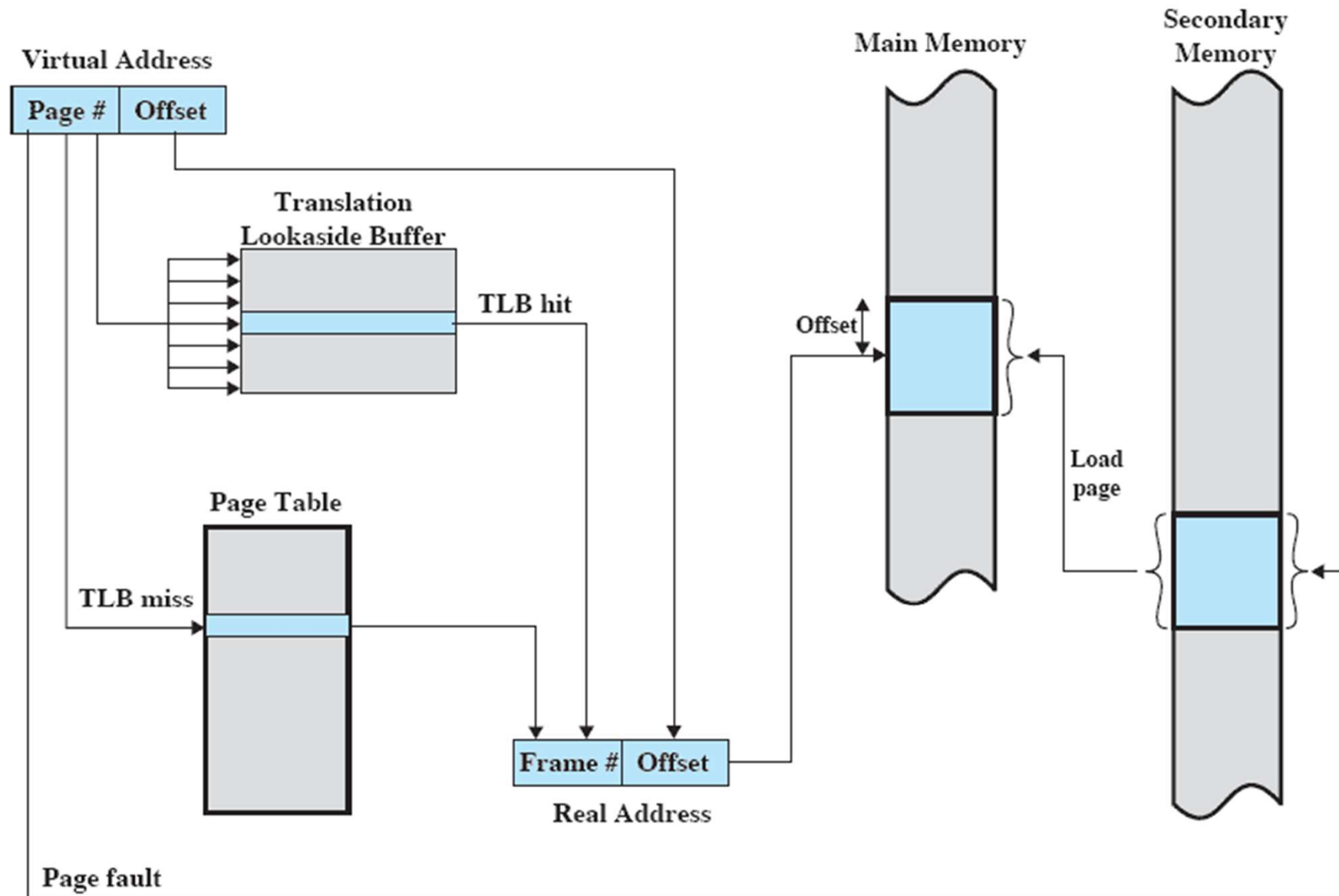
Translation Lookaside Buffer

- Each virtual memory reference can cause **two physical memory accesses** -- doubling access time
 - One to fetch the page table entry
 - One to fetch the data
- To overcome this problem a **high-speed cache** is set up for page table entries
 - Called a **T**ranslation **L**ookaside **B**uffer (**TLB**)
 - Contains **page table entries** that have been **most recently used**

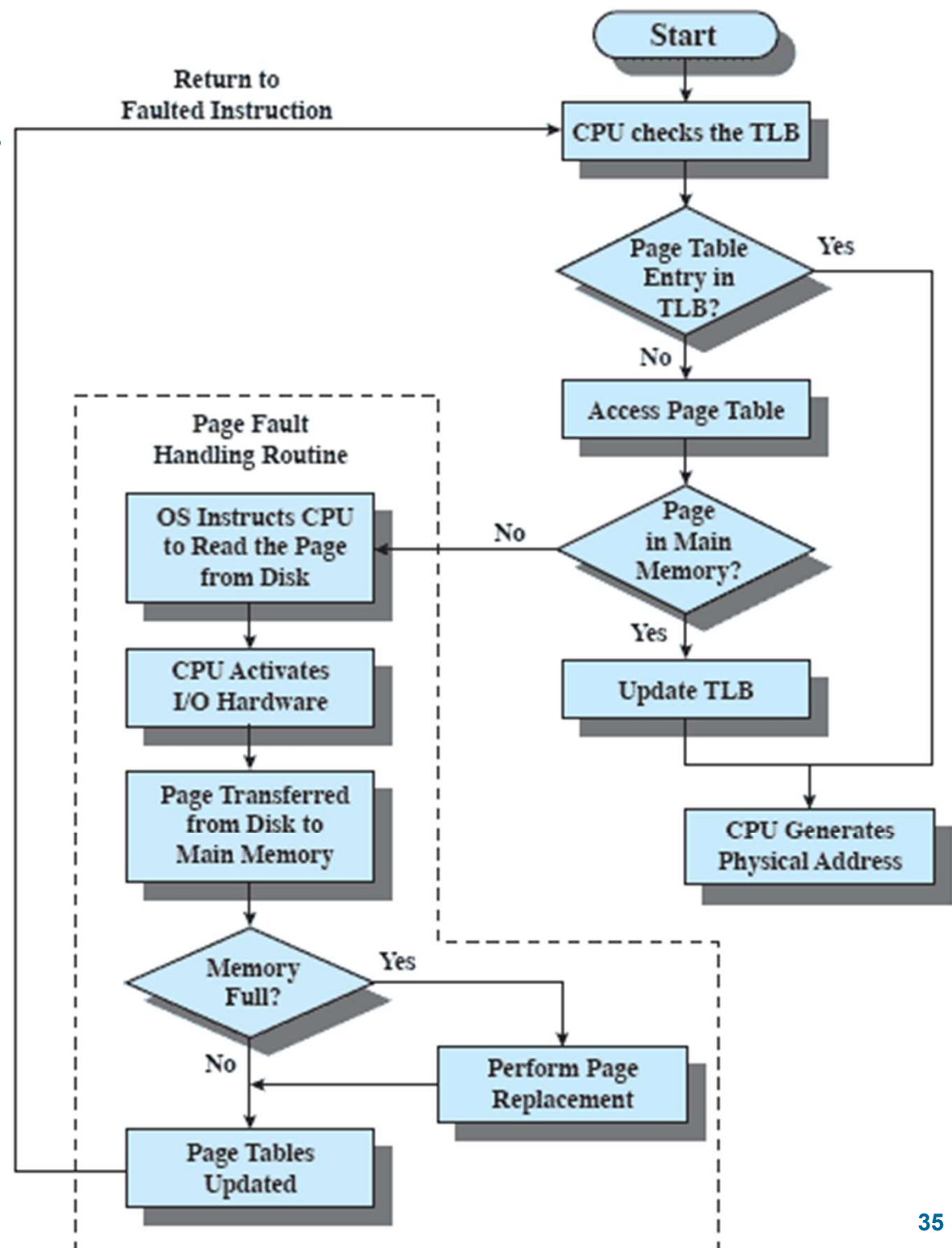
TLB Operation

- Given a virtual address
 - processor examines the TLB
- If page table entry is present (TLB hit)
 - the frame number is retrieved and the real address is formed
- If page table entry is not found in the TLB (TLB miss)
 - the page number is used to index the process page table
 - Processor checks if page is already in main memory
 - ▶ If not in main memory a page fault is issued
 - The TLB is updated to include the new page entry

Use of Translation Lookaside Buffer



TLB operation



Segmentation

- ❑ Segmentation allows programmer to view memory as consisting of multiple address spaces or segments
 - ❑ May be unequal, dynamic size
 - ❑ **Simplifies** handling of **growing** data structures:
 - ▶ Data can be assigned its own segment, to be expanded and shrunk by OS as needed
 - ▶ If a segment that needs to be expanded is in main memory and no room, the OS may move the segment to a larger area of main memory, if available, or swap it out. The enlarged segment would be swapped back in at the next opportunity
 - ❑ Allows programs to be altered/recompiled independently: without relinking and reloading the entire set of programs
 - ❑ Lends itself to **sharing** data among processes
 - ▶ Programmer can place a utility program or a table of data in a shared segment
 - ❑ Lends itself to **protection**: programmer assign privileges to segments

Segment Organization

- Starting address corresponding segment in main memory
- Segment Table Entry
 - Each entry contains the length of the segment
 - A bit is needed to determine if segment is already in main memory
 - Another bit is needed to determine if the segment has been modified since it was loaded in main memory

Virtual Address



Segment Table Entry



(b) Segmentation only

Address Translation in Segmentation

- register holds the starting address of the segment table
- segment number of a virtual address is used to index that table and look up the corresponding main memory address of that segment
- combined with the offset portion of the virtual address to produce the desired real address.

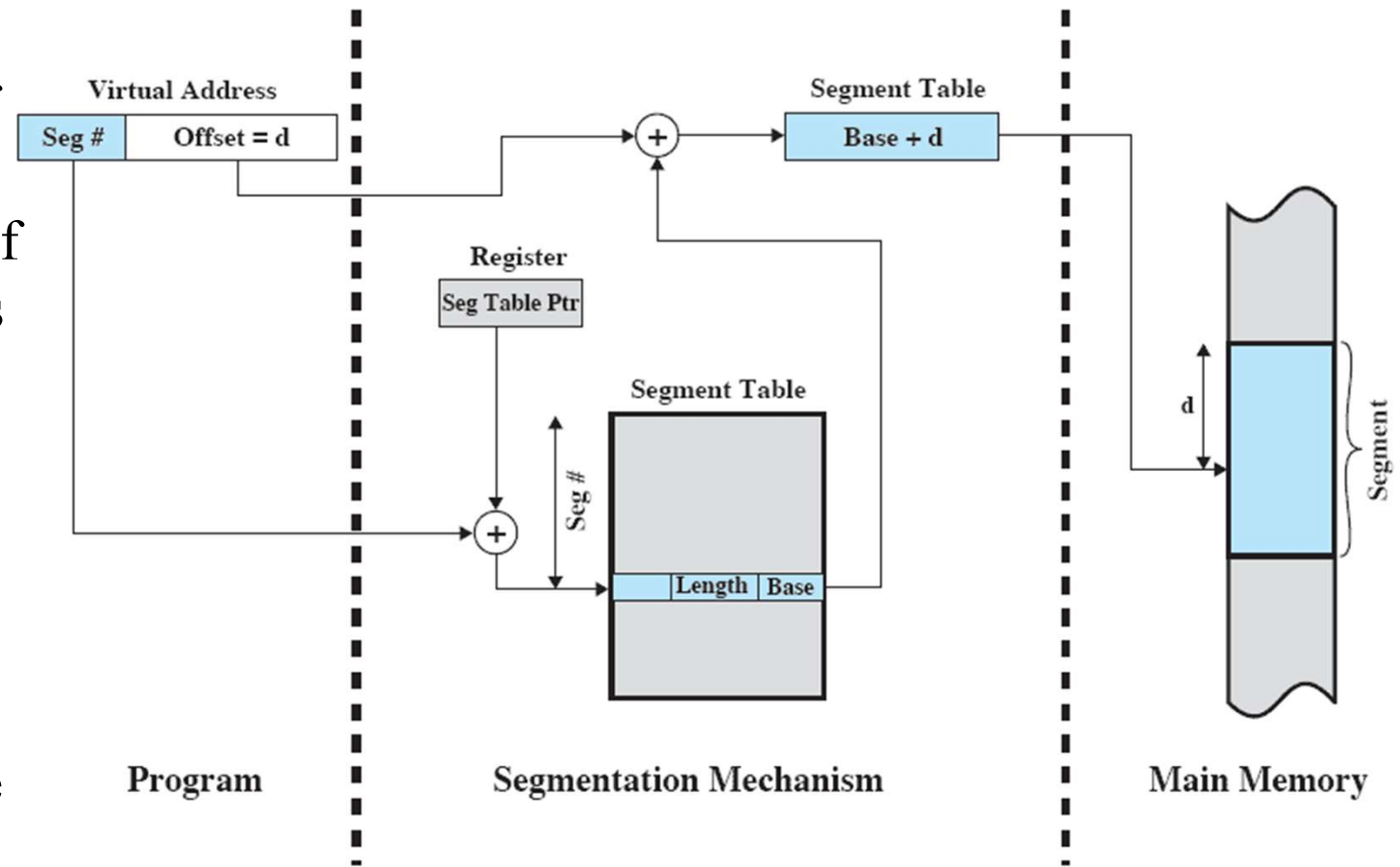


Figure 8.12 Address Translation in a Segmentation System

Protection and sharing

- ❑ Segmentation lends itself to the implementation of protection and sharing policies
- ❑ As each entry has a base address and length, inadvertent memory access can be controlled
- ❑ Sharing can be achieved by segments referencing multiple processes

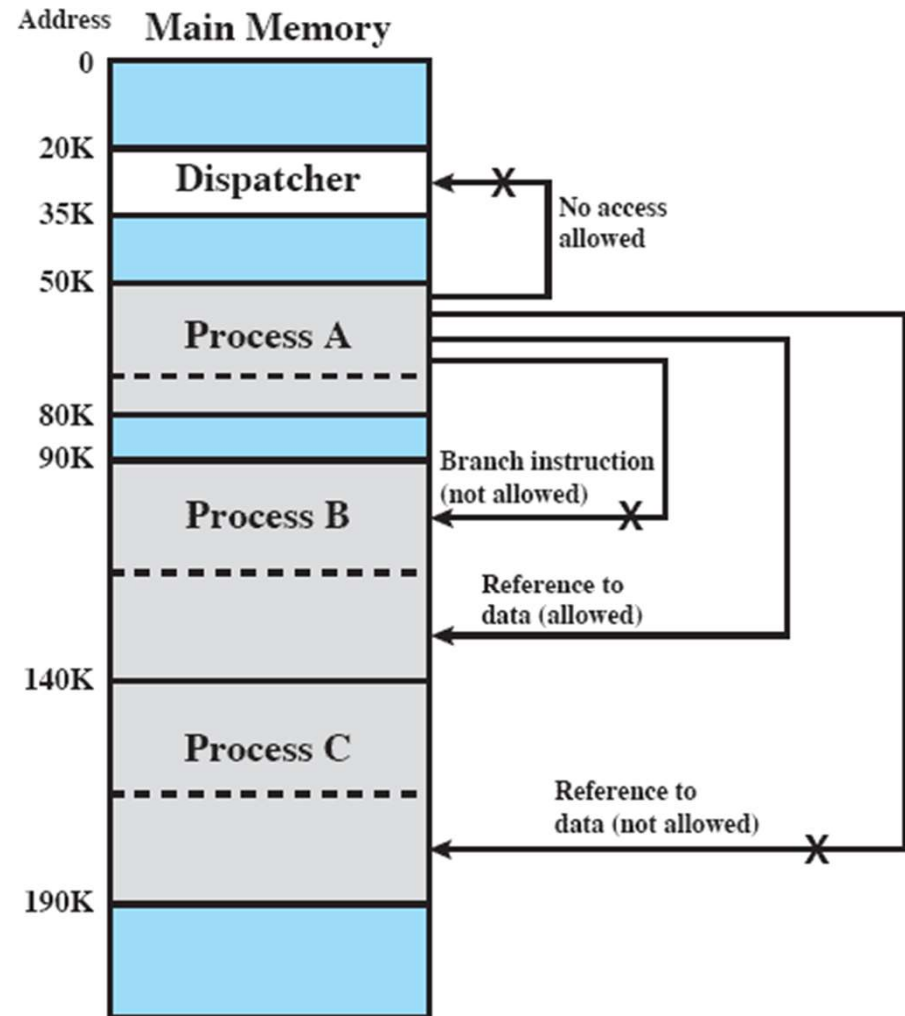


Figure 8.14 Protection Relationships Between Segments

Combined Paging and Segmentation

□ Paging

- transparent to the programmer
- no external fragmentation
- Fixed equal size allows for optimized MM algorithms

□ Segmentation

- visible to the programmer
- ability to handle growing data structures
- support for sharing and protection.

- To combine the advantages of both, some systems are equipped with processor hardware and operating system software to provide both.

Combined Paging and Segmentation

- A user's address space is broken into a number of segments, by the programmer
- Each segment is broken into a number of fixed-size pages, by OS
- For programmer: a logical address = segment # + segment offset.
- For the system: segment offset = page # + page offset within the specified segment

Virtual Address



Segment Table Entry



Page Table Entry



P= present bit

$M = M_0, M_1, \dots, M_{n-1}$

Address Translation

- Each process has a segment table and several page tables, one per process segment
- given a virtual address, the processor uses the segment # to index in the segment table to find the page table for that segment
- Then the page# is used to index the page table and look up the frame number
- This is combined with the offset portion of the virtual address to produce the desired real address

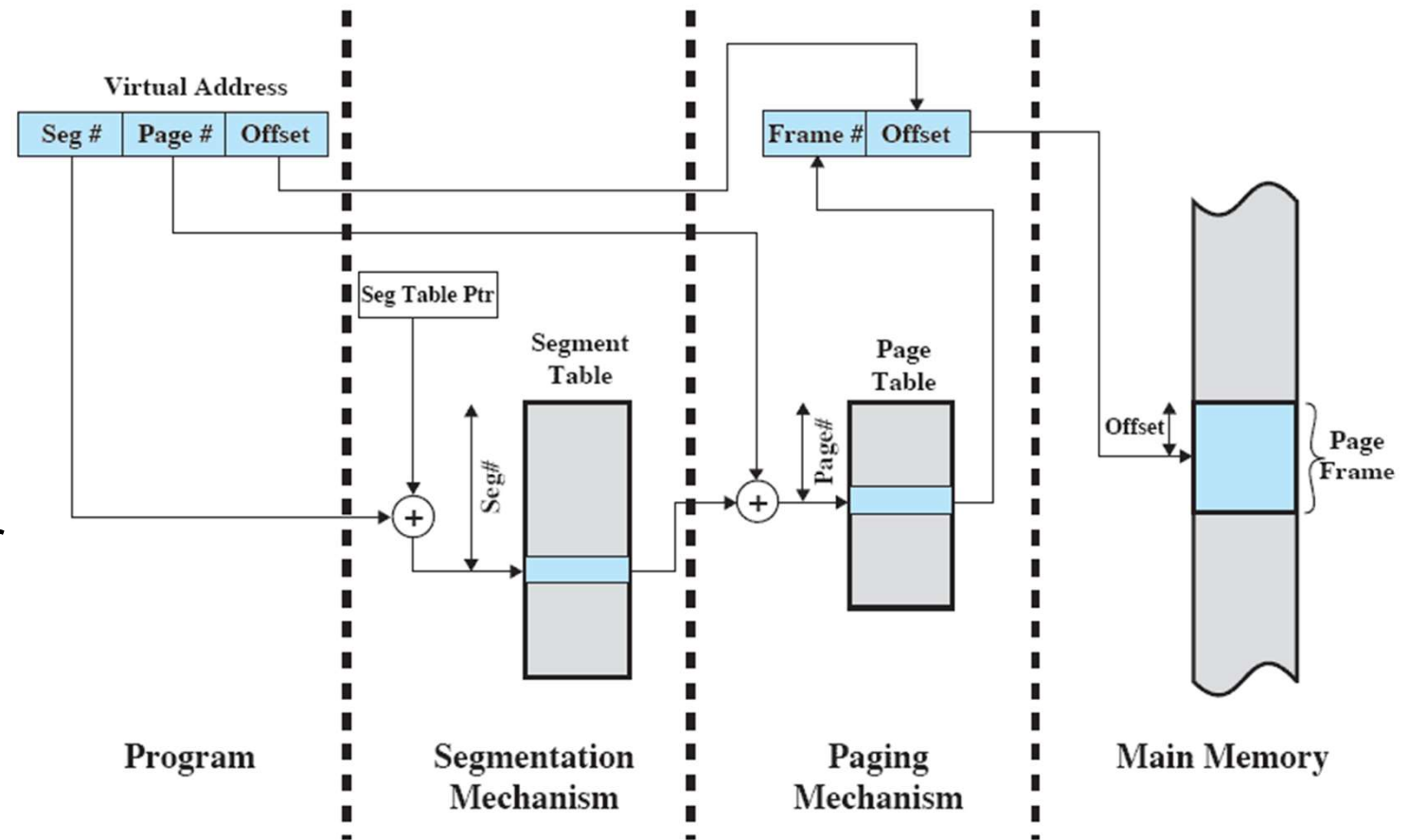


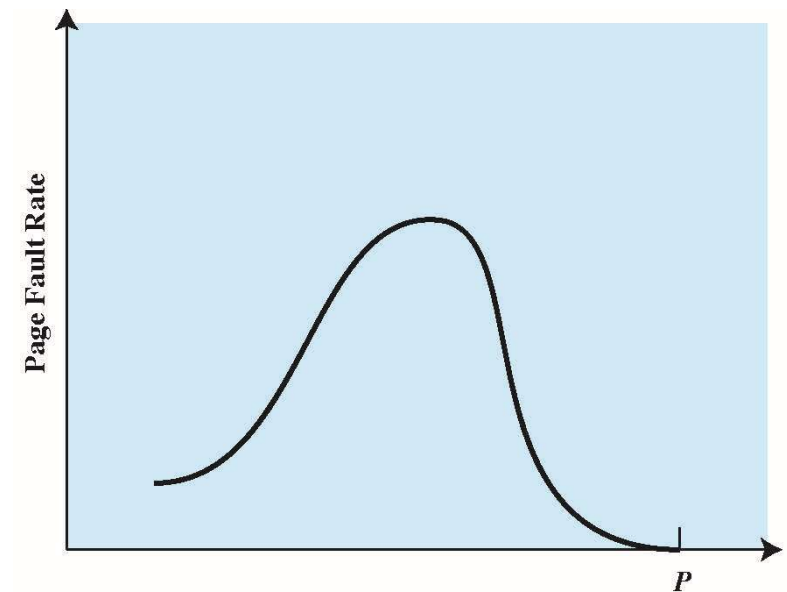
Figure 8.13 Address Translation in a Segmentation/Paging System

Page Size

- Smaller page size → less amount of **internal fragmentation** (due to the last page of a process)
- But -- Smaller page size
 - More pages required per process
 - Larger page tables
 - Page tables in virtual memory
(i.e. double page faults possible)
- Also, secondary memory is designed to efficiently transfer large blocks of data which favors a **larger page size**

Page Size – effect of faults rates

- Behavior is based on **principle of locality**
- **Small page size** → large # of pages will fit in main memory
 - the pages in memory will all contain portions of a process near recent references
 - → Page faults low
- **Increased page size** each page will contain locations further and further from any particular recent reference
 - Effect of locality is weakened
 - → Page faults high
- Thus locality of references favours small size pages
- the page fault rate will fall as the size of a page approaches the size of the entire process



(a) Page Size

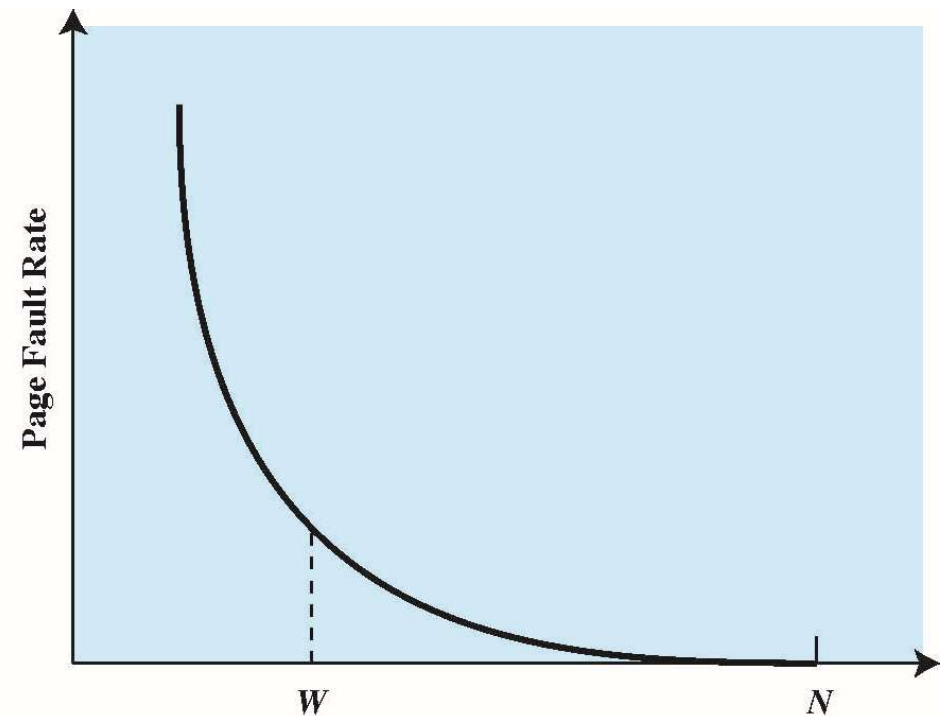
P = size of entire process

W = working set size

N = total number of pages in process

Page size – effect of # frames allocated

- the page fault rate is also determined by the number of frames allocated to a process
- for a fixed page size, the fault rate drops as the number of pages maintained in main memory grows
- Thus, a software policy (the amount of memory to allocate to each process) interacts with a hardware design decision (page size)



(b) Number of Page Frames Allocated

P = size of entire process

W = working set size

N = total number of pages in process

Page Size

- New programming techniques tend to decrease locality of references
 - OO: small programs and data modules scattered over a large number of objects
 - Multithreading: results in quick changes in instruction streams and scattered memory references
- locality of reference decrease → hit ratio on TLB accesses decrease
 - Use larger TLB – expensive
 - Use larger page size – performance degradation

Example Page Size

Computer	Page Size
Atlas	512 48-bit words
Honeywell-Multics	1024 36-bit word
IBM 370/XA and 370/ESA	4 Kbytes
VAX family	512 bytes
IBM AS/400	512 bytes
DEC Alpha	8 Kbytes
MIPS	4 Kbytes to 16 Mbytes
UltraSPARC	8 Kbytes to 4 Mbytes
Pentium	4 Kbytes or 4 Mbytes
IBM POWER	4 Kbytes
Itanium	4 Kbytes to 256 Mbytes

- ❑ Some architectures use multiple page sizes
- ❑ Multiple page sizes provide the **flexibility** needed to **use a TLB effectively**
 - ❑ large contiguous regions e.g. program instructions, may be mapped using a small number of large pages
 - ❑ thread stacks may be mapped using the small page size

Paging Performance

- Paging Time: **Page Fault Time**
 - Disk latency 8 milliseconds
 - Disk seek 15 milliseconds
 - Disk transfer time 1 millisecond
 - Total paging time ~25 milliseconds
- Could be longer due to
 - device queueing time
 - other paging overhead
- **Effective Access Time:**

$$EAT = (1 - p) \times ma + p \times pft$$

where: p is probability of page fault

ma is memory access time

pft is page fault time

Paging Performance (continued...)

- Effective access time with 100 ns memory access and 25 ms page fault time:

$$\begin{aligned}EAT &= (1 - p) \times ma + p \times pft \\&= (1 - p) \times 100 + p \times 25,000,000 \\&= 100 + 24,999,900 \times p\end{aligned}$$

- What is the EAT if $p = 0.001$ (1 out of 1000)?
 - $100 + 24,999,990 \times 0.001 = 25$ microseconds
 - 250 times slowdown!
- How do we get less than 10% slowdown?
 - $100 + 24,999,990 \times p \leq 1.10 \times 100 \text{ ns} = 110 \text{ ns}$
 - Less than 1 out of 2,500,000 accesses fault

Key aim: Minimise page faults

Operating Systems Software

□ Decisions in Memory Management

- The use of paging or segmentation or both – hardware support
- The algorithms employed for various aspects of MM
 - ▶ Key aim: Minimise page faults to reduce overhead

□ Page faults cause considerable overhead

- Deciding which page(s) to replace
- I/O exchanging pages
- Schedule another process
- Execute process switch

Decisions about Virtual Memory

❑ Fetch Policy

- ❑ When to bring a page in? When needed or in anticipation of need?

❑ Placement Where to put it? Unused page?

❑ Replacement What to unload to make room for a new page?

❑ Resident Set Management

- ❑ How many process pages to keep in memory? Fixed or variable?
- ❑ Reassign pages to other processes?

❑ Cleaning Policy

- ❑ When is a page written to disk?

❑ Load Control

- ❑ How many resident processes in memory?

Memory Management Policies

Fetch Policy

- Demand

- Prepaging

Placement Policy

Replacement Policy

- Basic Algorithms

 - Optimal

 - Least recently used (LRU)

 - First-in-first-out (FIFO)

 - Clock

- Page buffering

Resident Set Management

- Resident set size

 - Fixed

 - Variable

- Replacement Scope

 - Global

 - Local

Cleaning Policy

- Demand

- Precleaning

Load Control

- Degree of multiprogramming

No definitive best policy

Fetch Policy

- Determines when a page should be brought into memory
- **Demand paging**
 - only brings pages into main memory when a reference is made to a location on the page
 - Many page faults when process first started
 - principle of locality suggests that most future references will be to pages that have recently been brought in, so page faults should drop
- **Prepaging**
 - brings in more pages than needed
 - Exploits ccs of secondary storage: More efficient to bring in pages that reside contiguously on the disk
 - Ineffective if extra pages are not referenced
 - Can be employed either when a process starts or when a page fault occurs
 - Don't confuse with “swapping”: when process suspended all its resident pages are moved out. When resumed, all pages that were previously in main memory are returned to main memory.

Placement Policy

Determines **where** in real memory a process piece is to reside

- Pure segmentation: important decision
 - ▶ Best-fit, First-Fit, or Next-Fit can be used
- Pure paging or combined: trivial decision – can be placed anywhere
 - ▶ Address translation hardware and main memory access hardware can perform their functions for any page-frame combination with equal efficiency
- A concern with distributed shared memory systems
 - ▶ Memory can be referenced by any processor
 - ▶ Access time depends on distance between processor and memory module

Page Replacement

- Replacement policy: deals with the selection of a page in main memory to be replaced when a new page must be brought in
- Frame replacement
 1. select a frame (**victim**)
 2. write the victim frame to disk
 3. read in new frame from disk
 4. update page tables
 5. restart process

– **two page transfers**
- Reduce overhead using *dirty bit*
 - dirty bit is set whenever a page is modified
 - if dirty, write page, else just throw it out

Replacement Policy

- If memory is full, which page should be replaced?
 - Page that is least likely to be referenced in the near future
 - ▶ How is that determined?
 - ▶ Principal of locality again
 - Most policies predict future behavior based on past behavior
 - Tradeoff: the more sophisticated the policy, the greater hardware/software overhead to implement it
- Frame Locking
 - Associate a **lock** bit with each **frame**. If frame is locked, it may not be replaced
 - Kernel, Control structures, I/O buffers

Basic Replacement Algorithms

Algorithms for the selection of a page to replace

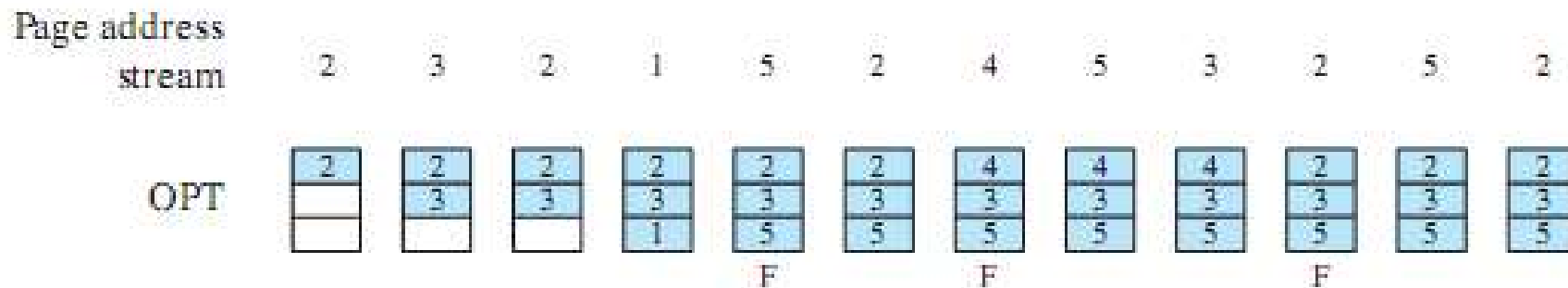
- **Random** (RAND)
 - ▶ choose any page to replace at random
- Belady's **Optimal** Algorithm
 - ▶ best but unrealizable – used for comparison
- First-In, First-Out (**FIFO**)
- Least Recently Used (**LRU**)
 - ▶ discard pages we have probably lost interest in
- **Clock** – Not Recently Used (NRU)
 - ▶ Efficient software LRU

Examples

- An example of the implementation of these policies will use a page address stream formed by executing the program is
 - 2 3 2 1 5 2 4 5 3 2 5 2
 - Which means that the first page referenced is 2,
 - the second page referenced is 3,
 - And so on.
- fixed frame allocation (fixed resident set size) for the process of **three** frames

Optimal Page Replacement Policy

- Selects the page for **which the time to the next reference is the longest**
- Results in fewest number of page faults
- Impossible to have perfect knowledge of future events
- Theoretical limit, not a practical algorithm
- Ex: assume fixed frame allocation of 3 frames
 - page address stream is 232152453252



F = page fault occurring after the frame allocation is initially filled

Produces **three page faults**

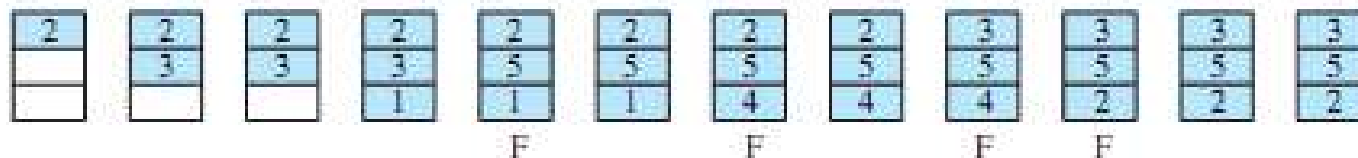
Least Recently Used (LRU)

- ❑ Replaces the page that has **not been referenced for the longest time**
- ❑ By the principle of locality, this should be the page least likely to be referenced in the near future
- ❑ Difficult to implement: Each page could be tagged with the time of last reference. **Large overhead!**
- ❑ LRU policy does nearly as well as the optimal policy

Page address
stream

2 3 2 1 5 2 4 5 3 2 5 2

LRU



F = page fault occurring after the frame allocation is initially filled

Four page faults

First-in, first-out (FIFO)

- Pages are removed in round-robin style – **replace oldest**
 - Simplest replacement policy to implement
- Page that has been **in memory the longest** is replaced
 - A page in memory a long time may have now fallen out of use
 - But, these pages may be needed again very soon
- Treats page frames of a process as a circular buffer

Page address
stream

1 3 2 1 5 2 4 5 3 2 5 2

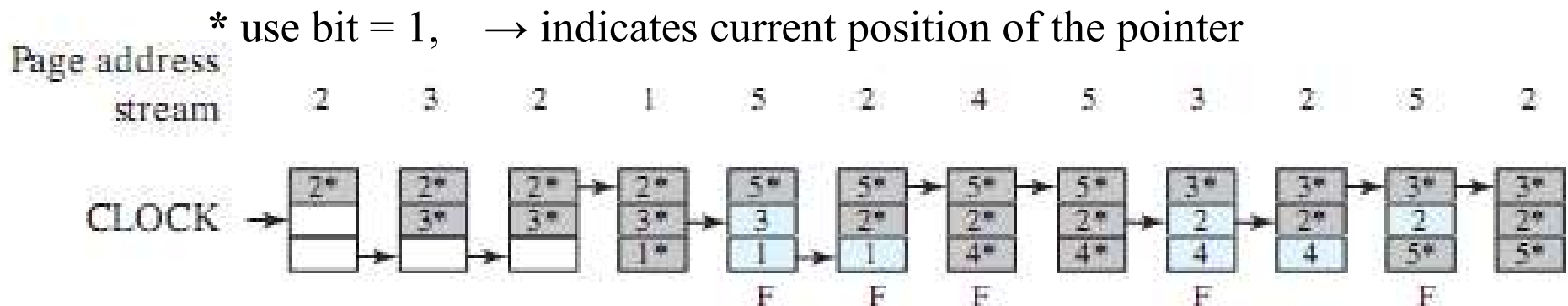
FIFO

2	2	2	2	5	5	5	5	3	3	3	3
	3	3	3	3	2	2	2	2	2	5	5
			1	1	1	4	4	4	4	4	2
				F	F	F		F		F	F

- FIFO results in **six page faults**.
 - Note that LRU recognizes that pages 2 and 5 are referenced more frequently than other pages, whereas FIFO does not.

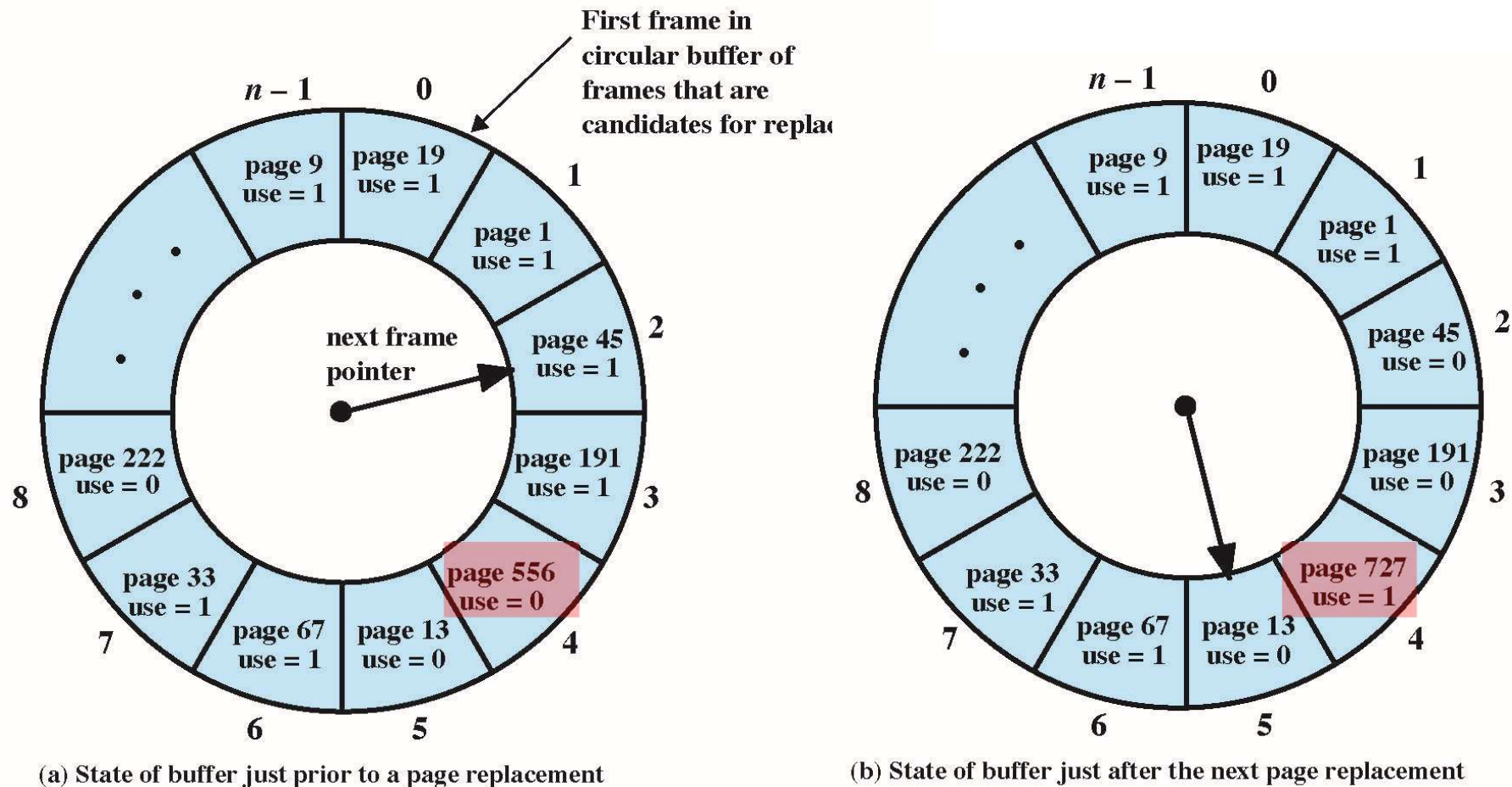
Clock Policy

- ❑ Uses an additional bit with each frame called a **use bit**
- ❑ When a page is first loaded in memory → **use bit = 1**
- ❑ When the page is referenced → **use bit = 1**
- ❑ Going clockwise with associated **pointer**
 - ❑ When a page is replaced, the pointer is set to the next frame
 - ❑ Scan the buffer: the first frame encountered with **use bit = 0** is replaced
 - ❑ During the search for replacement, **use bit** is changed from 1 to 0
 - ❑ If all frames have use bit=1, the pointer will make one complete cycle through the buffer, setting all the use bits to zero, and stop at its original position, replacing the page in that frame



- Clock ≈ FIFO but any frame with a use bit of 1 is passed over
- clock policy tries protecting frames 2 and 5 from replacement
- It tries to approximate LRU

Page 727 needs to be brought in, which page to replace?

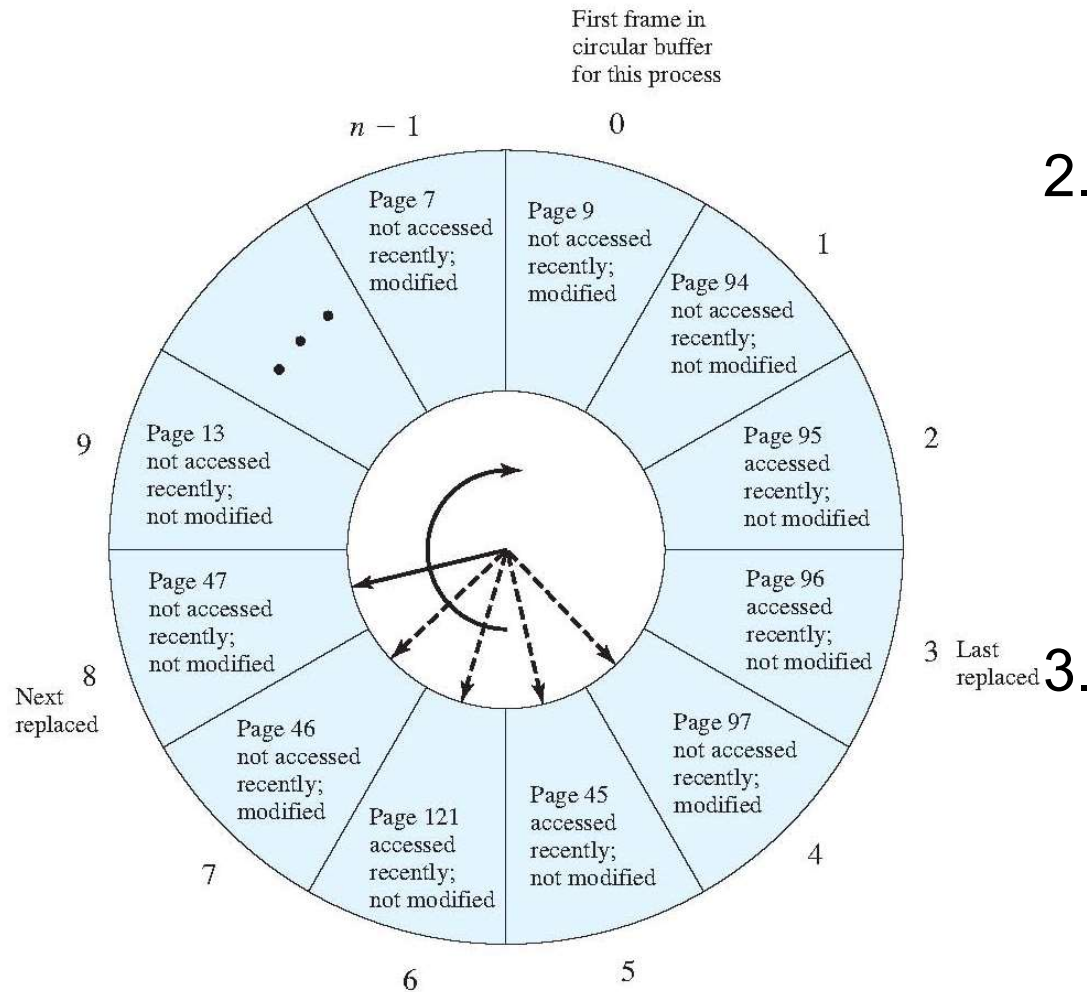


Example of a clock policy operation

Clock Policy - modified

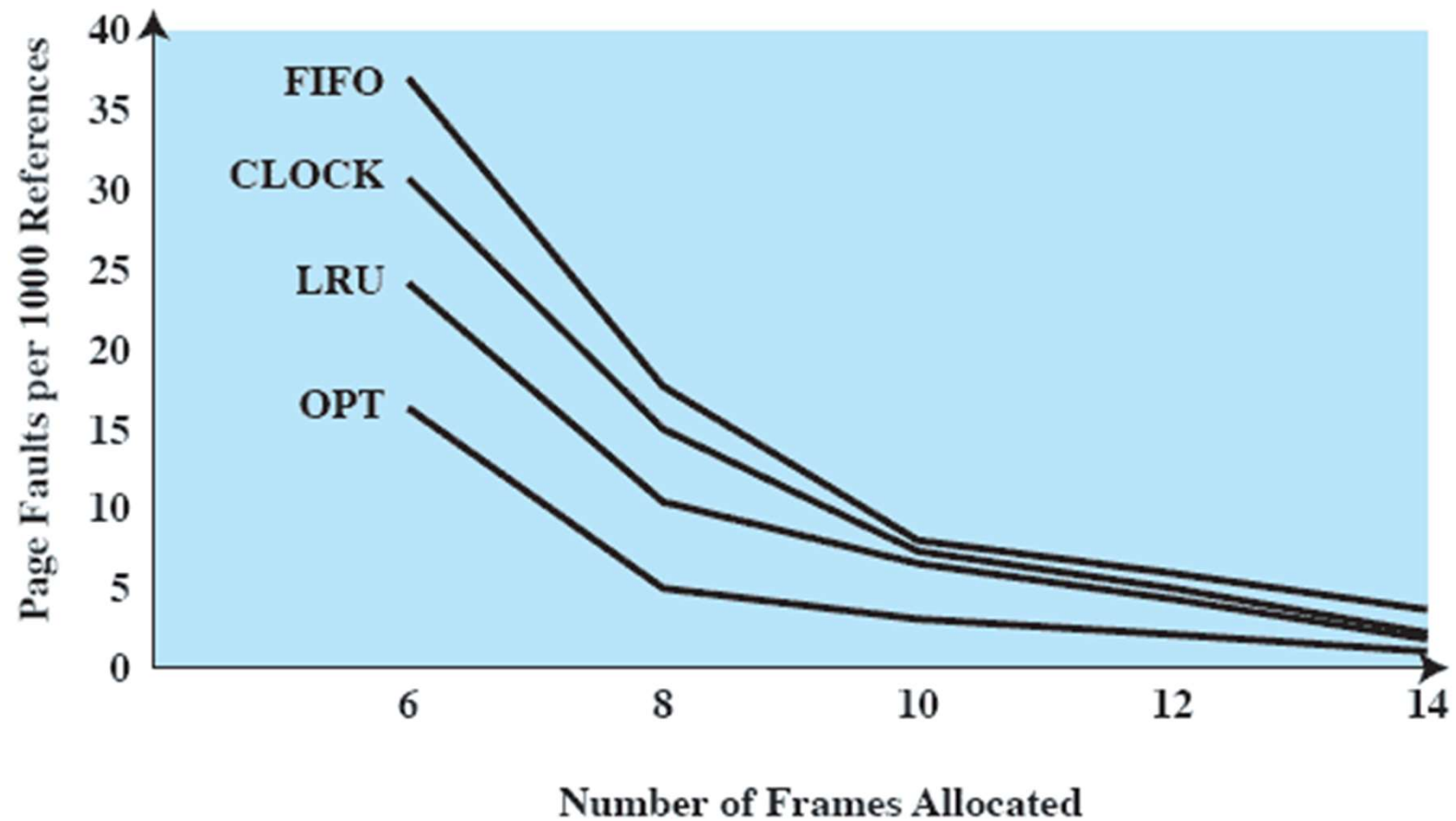
- ❑ Made more powerful by using extra bits
- ❑ Exploit **Modify bit**, associated with each frame/page signaling dirty page
- ❑ Taking **Modify** and **Use** bits, each frame is
 - ❑ Not accessed recently, not modified ($u=0;m=0$)
 - ❑ Accessed recently, not modified ($u=1;m=0$)
 - ❑ Not accessed recently, modified ($u=0;m=1$)
 - ❑ Accessed recently, modified ($u=1;m=1$)
- ❑ Algorithm cycles through pages in circular buffer looking for
 - ❑ Page **not modified** and **not accessed recently** – avoid writing it to disk
 - ❑ If no page found, make another cycle looking for a **modified page, not accessed recently**. Even though it has to be written to disk, but locality of reference suggests that it will not be needed again soon
 - ❑ If the 2nd pass fails, all frames are marked as not accessed and 3rd sweep is performed

Clock Policy - modified



1. scan the frame buffer, make no changes to the use bit. The first frame with ($u=0$; $m=0$) is selected for replacement
2. If step 1 fails, scan again, looking for the frame with ($u=0$; $m=1$). The first such frame encountered is selected for replacement. During this scan, set the use bit to 0 on each frame that is bypassed.
3. If step 2 fails, the pointer should have returned to its original position and all of the frames in the set will have a use bit of 0. Repeat step 1 and, if necessary, step 2. This time, a frame will be found for the replacement.

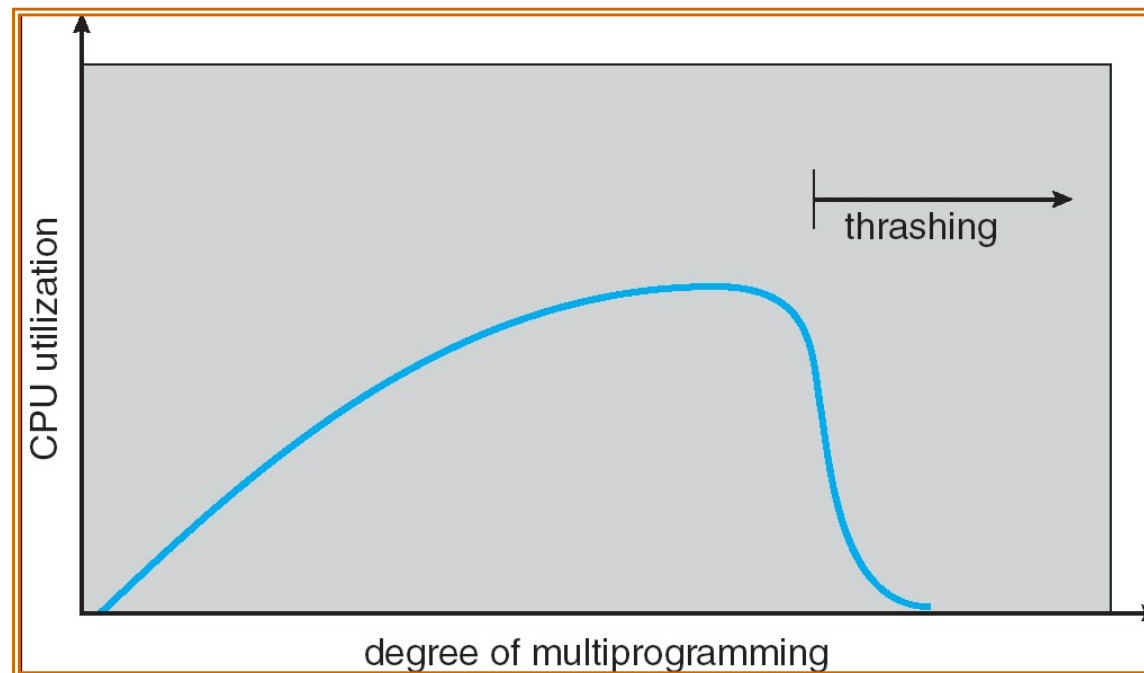
Comparison



Design Issues with Virtual Memory

Thrashing

- ❑ If a process does not have “enough” pages, the page-fault rate is very high. This leads to:
 - ❑ low CPU utilization
 - ❑ OS thinks it needs to increase the degree of multiprogramming
 - ❑ another process added to the system
- ❑ **Thrashing** ≡ a process is busy swapping pages in and out



Page Buffering

- Replacing a modified page is more costly than unmodified as needs written to secondary memory
- **Page Buffering**: a strategy to improve paging performance
 - Replacement algorithm is FIFO
 - A replaced page is added to one of two lists
 - ▶ **Free page list** if page has not been modified
 - ▶ **Modified page list**, if it has been modified
 - the page is not physically moved from main memory
 - ▶ Instead, the PTE for this page is removed and placed in free/modified page list
 - ▶ If the process references that page, it is returned to the resident set of that process at little cost.
 - ▶ In effect, the free and modified page lists act as a cache of pages.

Page Buffering

- The **free page list** is a list of page frames available for bringing in pages into memory
 - OS tries to keep some numbers of frames available at all times
 - When a page is to be read in, the page frame at the head of the list is used, destroying the page that was there
 - When an unmodified page is to be replaced, it remains in memory and its page frame is added to the tail of the free page list.
 - When a modified page is to be written out and replaced, its page frame is added to the tail of the modified page list.
- The **modified page list** serves another useful function:
 - Modified pages are written out in clusters rather than one at a time
 - significantly reducing the number of I/O operations and therefore the amount of disk access time

Paging Improvements

□ Working Set (Resident Set)

- a small number of pages per process are needed for execution (locality)
- most active set of pages should be in memory
 - Working Set

□ Pre-paging

- bring in pages that are likely to be used in the near future
- easier to guess at program startup
- may load unnecessary pages

Resident Set Management

- ❑ The OS must decide **how many pages** to bring into main memory
 - ❑ The smaller the amount of memory allocated to each process, the more processes that can reside in memory → CPU efficiency and less swapping
 - ❑ Small number of pages loaded increases page faults
 - ❑ Beyond a certain size, further allocations of pages will not affect the page fault rate, because of principle of locality

Resident Set Size

□ Fixed-allocation

- Gives a process a fixed number of frames within which to execute
- Size is decided at initial load/creation time
- When a page fault occurs, one of the pages of that process must be replaced

□ Variable-allocation

- Number of frames allocated to a process varies over the lifetime of the process
- A process suffering high level of page faults, indicating poor locality, will be given additional page frames
- A process with low page fault rates, indicating well behaved locality, will be given reduced allocation
- **Overhead**: requires OS to assess the behavior of active processes

Fixed Allocation

- **Equal allocation**: e.g, 100 frames and 5 processes, give each process 20 frames.
- **Proportional allocation** – Allocate according to the size of process

s_i = size of process p_i

$$S = \sum s_i$$

m = total number of frames

$$a_i = \text{allocation for } p_i = \frac{s_i}{S} \times m$$

$$m = 64$$

$$s_1 = 10$$

$$s_2 = 127$$

$$a_1 = \frac{10}{137} \times 64 \approx 5$$

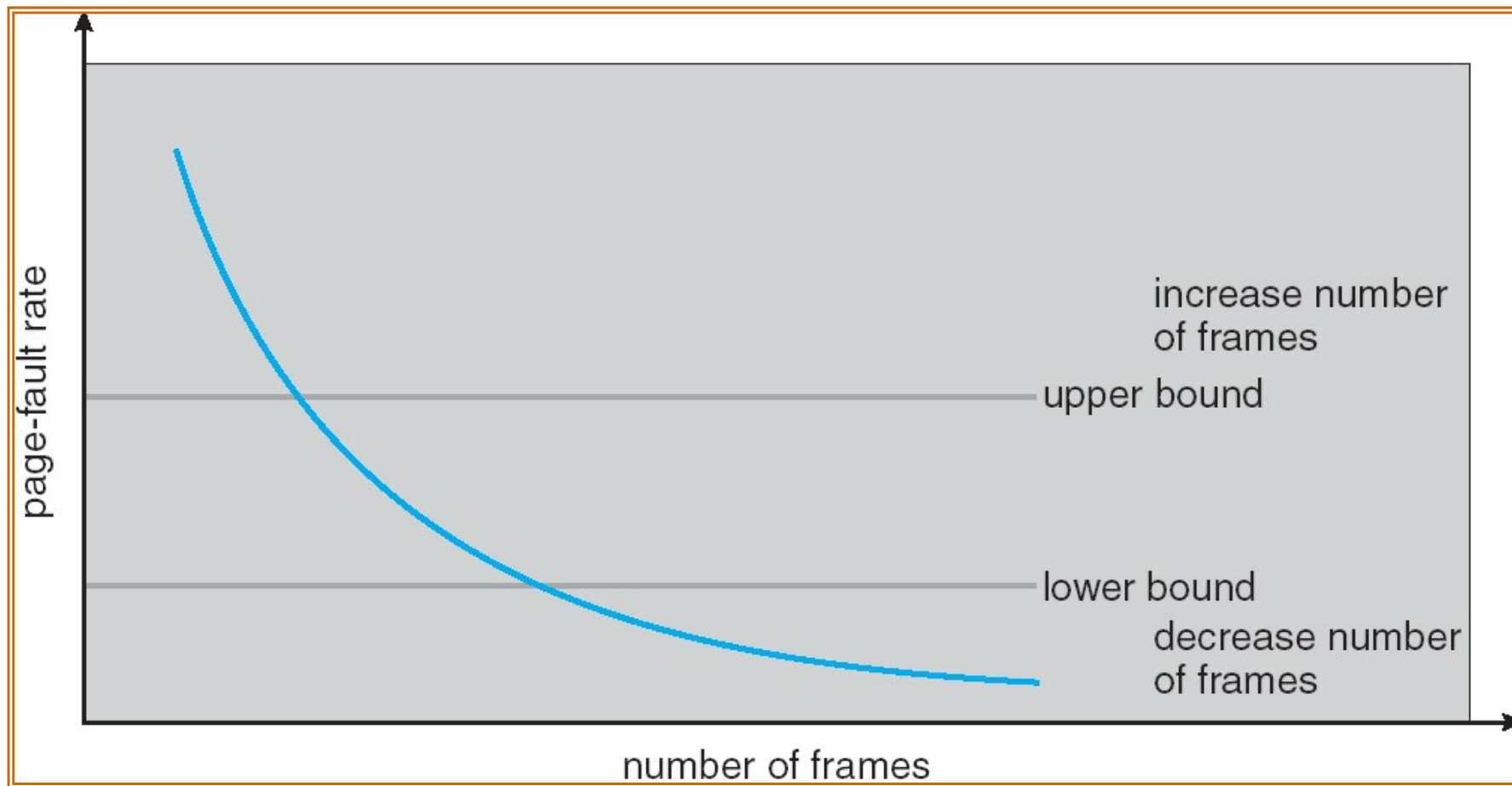
$$a_2 = \frac{127}{137} \times 64 \approx 59$$

Variable Allocation

- Use a proportional allocation scheme using priorities rather than size
- If process P_i generates a page fault,
 - select for replacement a frame from a process with lower priority number

Page-Fault Frequency Scheme

- Establish “acceptable” page-fault rate
 - If actual rate too low, process loses frame
 - If actual rate too high, process gains frame



Replacement Scope

- The scope of a replacement strategy can be categorized as *global* or *local*.
 - Both types are activated by a page fault when there are no free page frames.
 - A **Local Replacement Policy** chooses only among the resident pages of the process that generated the page fault in selecting a page to replace
 - A **Global Replacement Policy** considers all unlocked pages in main memory as candidates for replacement

Global vs. Local Allocation (cont)

Time last reference

Age	
A0	10
A1	7
A2	5
A3	4
A4	6
A5	3
B0	9
B1	4
B2	6
B3	2
B4	5
B5	6
B6	12
C1	3
C2	5
C3	6

(a)

A0
A1
A2
A3
A4
A6
B0
B1
B2
B3
B4
B5
B6
C1
C2
C3

(b)

A0
A1
A2
A3
A4
A5
B0
B1
B2
A6
B4
B5
B6
C1
C2
C3

(c)

- A, B, C processes in memory
- Process A page faults requesting A6.
- LRU used

Resident Set Management Summary

There is a correlation between replacement scope and resident set size

	Local Replacement	Global Replacement
Fixed Allocation	• Number of frames allocated to process is fixed. • Page to be replaced is chosen from among the frames allocated to that process.	• Not possible.
Variable Allocation	• The number of frames allocated to a process may be changed from time to time, to maintain the working set of the process. • Page to be replaced is chosen from among the frames allocated to that process.	• Page to be replaced is chosen from all available frames in main memory; this causes the size of the resident set of processes to vary.

Fixed Allocation, Local Scope

- Decide ahead of time the amount of allocation to give a process
 - Based on type of process (interactive, batch), amount requested by program or guidance
- Drawbacks
 - If allocation is too small, there will be a high page fault rate
 - If allocation is too large there will be too few programs in main memory
 - ▶ Increased processor idle time or
 - ▶ Increased swapping.

Variable Allocation, Global Scope

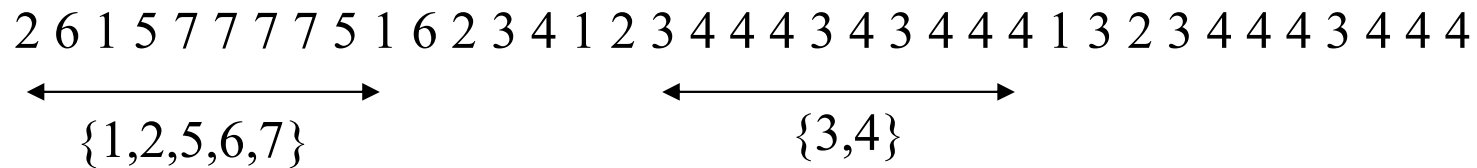
- Easiest to implement
 - Adopted by many operating systems
- Operating system keeps list of free frames
- Free frame is *added* to resident set of process when a *page fault* occurs
 - Process experiencing page fault will grow in size
- If no free frame, replaces one from another process
 - Choose a Replacement algorithm
 - Selection from all of the unlocked frames in memory
 - **Hard choice!** The process that suffers the reduction in resident set size may not be optimum
 - Use **page buffering**: choice of which page to replace becomes less significant, because the page may be reclaimed if it is referenced before the next time that a block of pages are overwritten.

Variable Allocation, Local Scope

- When a new process is added, allocate to it a number of page frames based on the application type, or other criteria (resident set)
- When page fault occurs, select page from the resident set of this process
- Reevaluate allocation
 - From time to time; increase/decrease the resident set size to improve performance
 - Decision is based on an assessment of the likely future demands of active processes
 - ▶ More complex, higher overhead
 - ▶ May yield better performance
 - Key issue is the criteria to decide the resident/working set size

Working Set

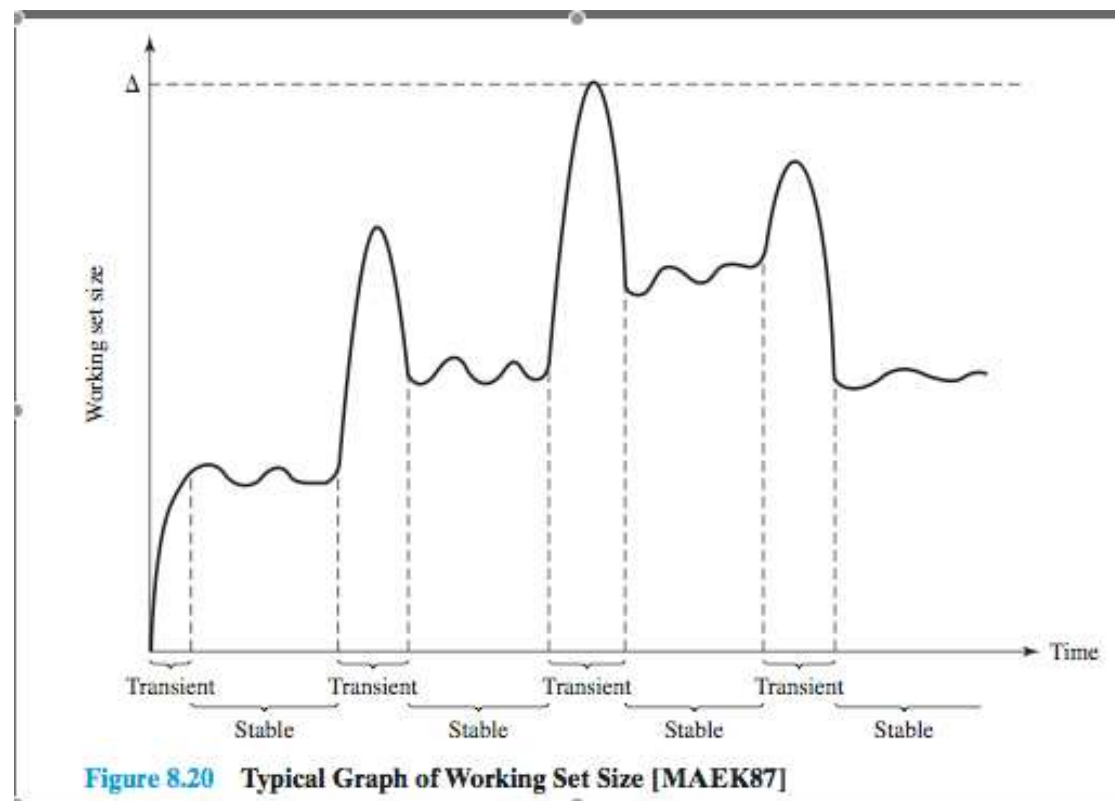
- The set of pages that are referenced within a time delta.
 - Approximation of program locality



- The working set window is **a moving window**. At each memory reference a new reference appears at one end and the oldest reference drops off the other end.
- Accuracy depends on time delta.
 - Too small → will not encompass entire locality
 - Too large → may contain several localities

Working Set of a Process

- $W(t, \Delta)$: The set of pages accessed over the last Δ time units at time t
- *Principle of locality* ensures that the working set changes slowly
- Processes go through phases
 - **Stable** → stays within stable working set
 - **Transient** → working set is rapidly changing

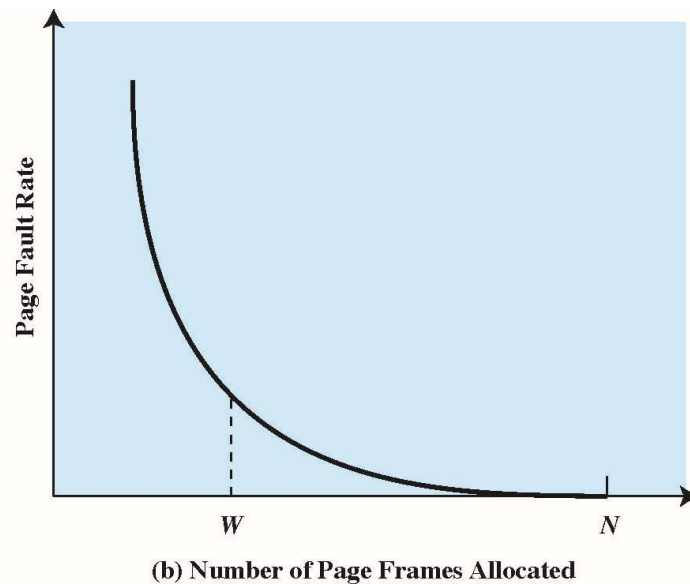


Resident Set Strategy

- The concept of W can guide a strategy for resident set size:
 - Monitor the working set of each process
 - Periodically remove from the resident set of a process those pages that are not in its working set. This is essentially an LRU policy
 - A process may execute only if its working set is in main memory (i.e., if its resident set includes its working set)
- This strategy is appealing because it exploits the principle of locality, which should minimize page faults
- **Problems:**
 - Past may not predict the Future:
 - ▶ size and membership of W will change over time
 - Hard to measure true working set
 - ▶ Need to time-stamp every page reference for every process and then maintain a time-ordered queue of pages for each process

Resident Set Strategy

- Approximate Resident Set Strategy, focusing on page fault rate, rather than on exact page references W



- Can try **varying set size** based on page fault frequency
 - If rate $<$ threshold₁, safe to remove pages from resident set
 - If rate $>$ threshold₂, add more pages to resident set

Resident Set Strategy

- Page-Fault Frequency (PFF) Algorithm
 - Associate **use bit** with each page, =1 when page is accessed
 - When page fault occurs, record time since last page fault **T**
 - If $T < F_1$: add page to the resident set
 - If $T > F_2$:
 - ▶ discard all pages with use bit=0 → shrink resident set
 - ▶ Set use bit = 0 for the remaining pages of the process
 - **Problem**: Doesn't handle transient periods well
 - ▶ The rapid rate of page faults during transient causes the resident set of a process to swell before the pages of the old locality are expelled;
 - ▶ may produce unnecessary process deactivations and reactivations, and switching and swapping overheads.

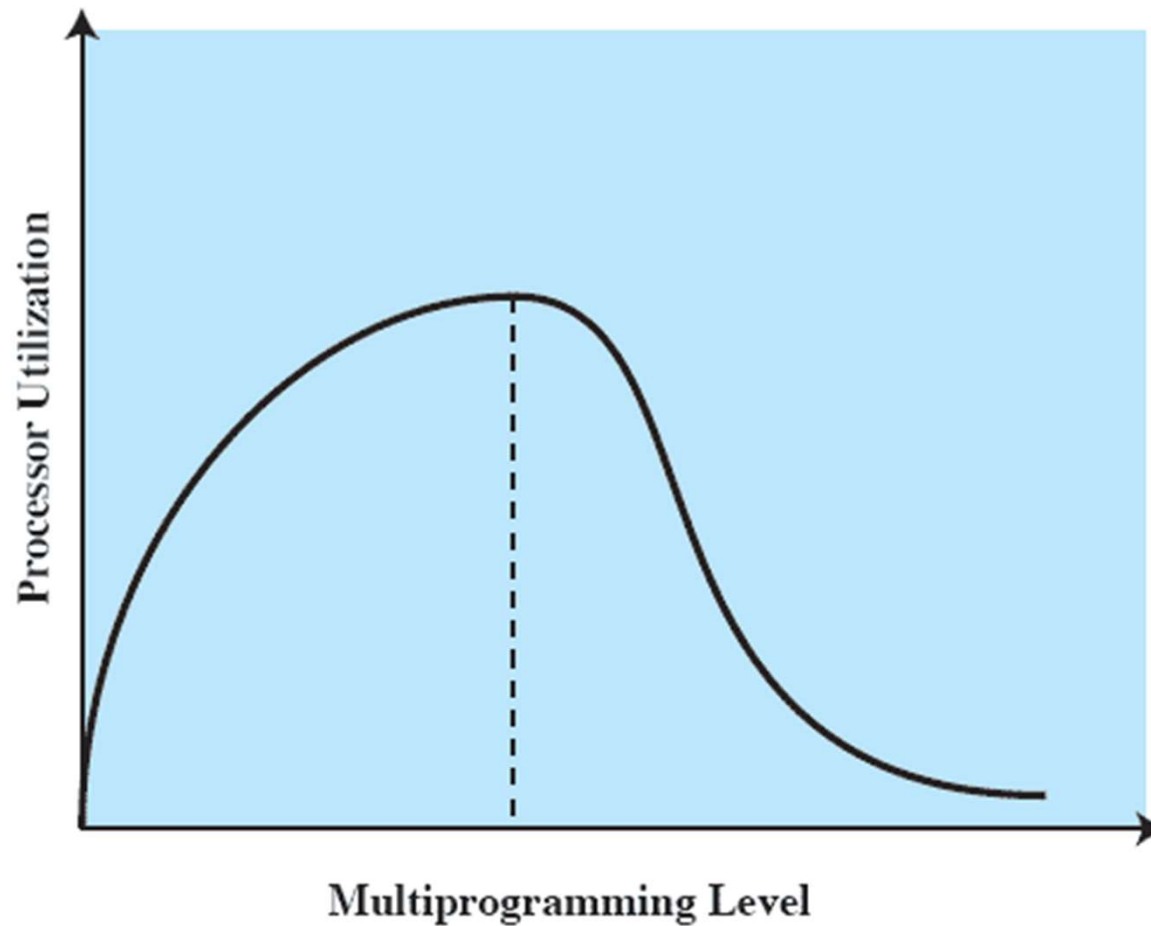
Cleaning Policy

- A cleaning policy: determine when a modified page should be written out to secondary memory
- Demand cleaning
 - A page is written out only when it is selected for replacement
- Precleaning
 - Pages are written out in batches
- Best approach uses page buffering
 - Replaced pages are placed in two lists: Modified & unmodified
 - Pages in Modified list are periodically written out in batches
 - Pages in the Unmodified list are either reclaimed if referenced or lost when its frame is assigned to another page

Load Control

- Determines the number of processes that will be resident in main memory: *multiprogramming level*
- As *multiprogramming level* increases, processor utilization rises, but
 - Too many processes
 - small *resident set* for each process
 - increase page faults → thrashing
 - Too few processes
 - higher probability for all processes to be blocked
 - idle CPU time increases
 - time spent in swapping

Multiprogramming Effects



Process Suspension

- If the degree of multiprogramming is to be reduced, some processes must be suspended (swapped out)
 1. **Lowest priority process**
 - Scheduling policy; unrelated to performance issue
 2. **Faulting process** (will be blocked anyway)
 - This process does not have its working set in main memory
 3. **Last process activated**
 - This process is least likely to have its working set resident
 4. **Process with smallest resident set**
 - This process requires the least future effort to reload
 - Penalize programs with strong locality
 5. **Largest process**
 - generates the most free frames reducing future suspensions
 6. **Process with the largest remaining execution window**

Other Issues – Program Structure

- `int[128,128] data;`
- Each row is stored in one page
- **Program 1**

```
for (j = 0; j < 128; j++)  
    for (i = 0; i < 128; i++)  
        data[i, j] = 0;
```

128 x 128 = 16,384 page faults

- **Program 2**

```
for (i = 0; i < 128; i++)  
    for (j = 0; j < 128; j++)  
        data[i, j] = 0;
```

128 page faults